

Universidad Alfonso X el Sabio

Business Tech

Máster Universitario en Inteligencia Artificial

Trabajo de Fin de Máster

Impacto de la Arquitectura RAG en las Alucinaciones de los LLMs

Autor: Samuel Lorenzo Sánchez

Tutor: Carlos Granados Aguilar

2025–2026

Índice general

1. Problema, contexto y utilidad real	2
1.1. Justificación del Proyecto	2
1.2. Contexto del Proyecto	4
1.3. Presentación del proyecto y del problema	5
1.4. Utilidad real de la propuesta	6
1.5. Estructura del trabajo	6
1.6. Estructura del trabajo	6
2. Objetivos y planificación	8
2.1. Objetivo general	8
2.2. Objetivos específicos	9
2.3. Planificación Scrum y lista de tareas	9
2.3.1. Tarea 1. Documentación, investigación y análisis del estado del arte	10
2.3.2. Tarea 2. Diseño del prototipo, las entidades y el dominio	10
2.3.3. Tarea 3. Diseño y creación del back-end	11
2.3.4. Tarea 4. Diseño y creación del front-end	11
2.3.5. Tarea 5. Módulo de trazabilidad y citación automática	11
2.3.6. Tarea 6. Evaluación del rendimiento y de la adopción	12
2.3.7. Tarea 7. Redacción de la memoria y presentación final	12
2.4. Correspondencia entre objetivos y tareas	12
3. Marco teórico / Estado del arte	14
3.1. Arquitectura Transformers y Grandes Modelos del Lenguaje	14
3.1.1. Fundamentos Matemáticos de los Transformers y Mecanismos de Atención	17
3.1.2. Limitaciones Estructurales de los LLMs	18
3.1.3. Taxonomía del Problema de las Alucinaciones en Dominios Normativos	19
3.1.4. Restricciones en la Ventana de Contexto y el Fenómeno Lost in the Middle	20
3.1.5. Volatilidad Normativa y Falta de Determinismo y Trazabilidad . . .	20
3.2. Arquitectura Retrieval-Augmented Generation (RAG)	21

3.2.1.	Descomposición del Pipeline de RAG: Indexación, Recuperación y Generación	22
3.3.	Revisión Crítica de la Arquitectura RAG en Dominios Regulados	26
3.4.	Alucinaciones en modelos de lenguaje en entornos normativos (definición, causas y estudios relevantes).	27
3.5.	Métricas de evaluación en sistemas RAG (definición formal de RAGAS: fidelidad, relevancia de respuesta, precisión y exhaustividad del contexto, tasa de alucinaciones, calidad de citación).	28
4.	Marco metodológico	30
4.1.	Arquitectura hexagonal como método de construcción del núcleo RAG . . .	30
4.2.	Población y muestra	33
4.3.	Objetivos de investigación	34
4.4.	Variables intervinientes e instrumentos de evaluación	34
4.5.	Diseño experimental	35
4.5.1.	Variables intervinientes e Instrumentos de Evaluación	35
4.6.	Modelo de análisis de datos	37
4.7.	Configuración experimental del prototipo	38
4.8.	Dataset de evaluación y categorías	38
4.9.	Origen de las métricas: propias, juez y RAGAS	39
5.	Arquitectura e implementación	40
5.1.	Diseño e implementación de la aplicación	40
5.2.	Back-end, diseño e implementación	40
5.2.1.	Capa de dominio	43
5.2.2.	Entidades	43
5.2.3.	Objetos valor	44
5.2.4.	Enumerados	45
5.2.5.	Capa de aplicación	45
5.2.6.	Capa de adaptadores	45
5.2.7.	Flujo end-to-end de una consulta	46
5.3.	Capturas del prototipo AgroPS	48
5.4.	Alcance multimodal del prototipo	48
6.	Despliegue operativo del prototipo contenedorizado	51
6.1.	Despliegue operativo del prototipo funcional contenedorizado	51
6.2.	Despliegue Operativo y Arquitectura de Contenedores (Docker)	51
7.	Evaluación, resultados y discusión	53
7.1.	Análisis descriptivo de la muestra	53

7.2. Resultados del recuperador	53
7.3. Calidad de generación (LLM-as-a-Judge)	55
7.4. Comparación LLM sin RAG frente a LLM con RAG	55
7.5. Resultados por categoría (juez y alucinación)	55
7.6. Adaptador RAGAS	55
7.7. Fallos en consultas de amplitud	57
7.8. Concordancia experto-juez	57
7.9. Discusión	57
8. Conclusiones	58
8.1. Limitaciones	59
8.2. Prospectiva	60
8.3. Consideraciones finales	60
A. Glosario	62
Bibliografía	66

Índice de figuras

3.1. Arquitectura Transformer (codificador–decodificador) con atención multi-cabeza, conexiones residuales y normalización (Vaswani et al., 2017).	15
3.2. Línea temporal de modelos de lenguaje, desde n-gramas y RNN/LSTM hasta Transformer, BERT, GPT y los LLM contemporáneos.	16
3.3. Desvanecimiento del gradiente en retropropagación: la magnitud de la señal decae al alejarse del instante actual, limitando a las RNN clásicas.	17
3.4. Matriz de autoatención sobre la oración «El banco de madera estaba en el parque»: cada consulta pondera las claves del contexto. Ilustra por qué el sentido de un token depende de su entorno y no de un embedding estático.	18
3.5. Hiperplano de decisión lineal $W \cdot X + b = 0$ en un espacio de dos atributos. Sirve de analogía geométrica a la separación de representaciones, previa a los espacios densos de los embeddings.	19
3.6. Fenómeno <i>Lost in the Middle</i> : la retención de un dato clave cae en el centro del prompt y se recupera en los extremos (primacía y recencia).	21
3.7. Pipeline RAG: la pregunta dispara una consulta de recuperación; los textos recuperados se concatenan al prompt completo y el LLM genera la respuesta anclada.	22
4.1. Mapeo de la arquitectura hexagonal al caso práctico AgroPS: dominio, aplicación y adaptadores de entrada y salida sobre el bounded context RAG.	31
5.1. Diagrama de la aplicación AgroPS: frontend, backend, base de datos e inferencia orquestados con Docker Compose.	41
5.2. Esquema del back-end AgroPS: dominio en el centro, aplicación con casos de uso y puertos, y tres adaptadores en el anillo exterior (API REST, PostgreSQL y Ollama).	41
5.3. Esquema del back-end hexagonal (<i>gateway/rag</i>). El lado conductor entra por <i>Adapter In / Port In</i> ; el lado conducido se conecta por <i>Port Out / Adapter Out</i>	42

5.4.	Entidades y objetos valor del dominio RAG: la consulta produce un RetrievalBundle que contiene uno o más RetrievedChunk , cada uno identificable por documento y fuente.	43
5.5.	Flujo end-to-end de una consulta en AgroPS: de la pregunta en el chat a la respuesta citada, pasando por FastAPI, la aplicación RAG, la recuperación híbrida y la generación local.	47
5.6.	Plataforma de gestión documental: listado de corpus PAC/POSEI, base de conocimiento, estado de indexado y número de chunks.	48
5.7.	Gestión de la organización: cooperativas y departamentos con aislamiento multi-tenant y roles de administrador, técnico e inspector.	49
5.8.	Componente de chat RAG: pregunta en lenguaje natural, respuesta anclada y panel de evidencia recuperada con puntuación y modo híbrido RRF. . . .	49
5.9.	Wizard de datasets e ingesta: schema mapping, preview tabular y banco de evaluación con las siete categorías normativas.	50
5.10.	Información representada en grafo: entidades normativas (PAC, POSEI, BCAM, ayudas) y relaciones extraídas del corpus.	50
7.1.	MRR del recuperador híbrido por categoría, con la línea global 0,7911. . .	54
7.2.	Comparación visual del LLM autónomo frente a AgroRAG en exactitud, exhaustividad, relevancia, fidelidad y tasa de alucinación.	56

Índice de tablas

2.1. Correspondencia entre objetivos específicos (épicas) y las siete tareas del MVP.	12
3.1. Comparación de arquitecturas y enfoques NLP relevantes para este trabajo.	14
3.2. Justificación de <code>qwen3-embedding:latest</code> frente a Sentence Transformers, BGE y E5, y efecto esperado sobre la recuperación semántica.	25
4.1. Configuración exacta verificable del prototipo AgroPS (valores del repositorio; digests de modelos pendientes de congelar en el acta de evaluación). . .	38
4.2. Siete categorías del dataset versionado, conteo y ejemplo representativo. . .	39
5.1. Correspondencia entre anillos del hexágono y paquetes del backend.	42
5.2. Entidades y objetos valor del dominio RAG.	44
5.3. Entidades de persistencia del back-end (SQLAlchemy) y sus relaciones principales.	44
7.1. Desempeño del recuperador por categoría de consulta.	54
7.2. Comparación principal: LLM autónomo frente a AgroRAG ($N = 21$). . . .	55
7.3. Resultados por las siete categorías del banco versionado.	56

Resumen

El presente Trabajo de Fin de Máster evalúa la capacidad de la arquitectura Generación Aumentada por Recuperación (RAG) para **mitigar** la tasa de alucinaciones y mejorar la exactitud factual en Modelos de Lenguaje de Gran Escala (LLM) aplicados a documentación normativa y técnica del sector agrícola (PAC y POSEI), en el marco del prototipo AgroPS.

Se implementó un **prototipo funcional contenedorizado** con tubería de recuperación híbrida (búsqueda densa sobre PostgreSQL/pgvector, BM25 y reordenador Cross-Encoder). El corpus se procesó con análisis de maquetación y OCR para estructurar tablas y jerarquías legales.

Para la auditoría experimental se versiona un banco de evaluación de **21 ítems (19 preguntas únicas)** en siete categorías; la ampliación a 150 casos con validación experta documentada permanece como trabajo pendiente. La evaluación separa: (i) métricas propias de retrieval, (ii) LLM-as-a-Judge y (iii) adaptador RAGAS con métrica binaria `is_hallucination`, sin mezclar familias de métricas.

Los resultados exploratorios se resumen así: MRR global de 0,7911, exactitud media del juez de 4,21 sobre 5 y tasa de alucinación binaria de 0,19 con RAG frente a 0,62 sin recuperación. Las categorías holísticas y de amplitud normativa degradan el recuperador (MRR 0,518 y 0,470). El prototipo incluye ingesta documental, asistente RAG con citas y laboratorio de evaluación.

Abstract

This Master's Thesis evaluates Retrieval-Augmented Generation (RAG) for **mitigating** factual hallucinations in LLMs applied to agricultural regulatory documentation (CAP/POSEI), within the AgroPS containerized prototype. Hybrid retrieval combines dense search on PostgreSQL/pgvector, BM25 and a Cross-Encoder reranker. The evaluation bank contains 21 items (19 unique questions) across seven categories. Reported scores include global MRR 0.7911, judge accuracy 4.21/5 and binary hallucination 0.19 with RAG versus 0.62 without retrieval.

Palabras clave

Generación Aumentada por Recuperación (RAG), Alucinaciones, NLP, LLM, IA Generativa, PostgreSQL/pgvector, Evaluación RAGAS.

Keywords

Retrieval-Augmented Generation (RAG), AI Hallucinations, NLP, LLM, Generative AI, PostgreSQL/pgvector, RAGAS evaluation.

Capítulo 1

Problema, contexto y utilidad real

Este Trabajo de Fin de Máster presenta el diseño y desarrollo de un sistema de asistencia documental basado en Generación Aumentada por Recuperación aplicado a la documentación agrícola y normativa. El objetivo principal consiste en reducir la aparición de alucinaciones en las respuestas generadas por modelos largos del lenguaje (LLM), incorporando el mecanismo de recuperación de información que permita fundamentar las respuestas en fuentes documentales concretas y trazables. (Huang, 2023)

En este capítulo se expone, en primer lugar, la justificación del proyecto y el contexto en el que se sitúa. A continuación, se presenta el problema que motiva la investigación y se describe la utilidad práctica de la solución propuesta. Finalmente, se introduce la estructura general del trabajo para orientar la lectura de los capítulos posteriores.

1.1. Justificación del Proyecto

La elección de este proyecto responde a una necesidad muy concreta dentro del uso actual de la inteligencia artificial generativa: la de disponer de sistemas capaces de proporcionar respuestas útiles, pero también verificables, especialmente cuando se trabaja con documentación técnica y normativa. Los modelos de lenguaje de gran escala han transformado la manera en que se accede a la información, ya que permiten consultar datos mediante lenguaje natural y generar respuestas fluidas en cuestión de segundos. Sin embargo, su utilización también ha puesto en evidencia un problema de fondo: la generación de información incorrecta o no sustentada, conocida como alucinación. La literatura reciente sigue señalando este fenómeno como una de las principales barreras para el despliegue fiable de los LLMs en escenarios reales. (Lewis P. P., 2020)

En contextos generales, una alucinación puede percibirse como un error más dentro del comportamiento del modelo. No obstante, en ámbitos donde la precisión factual es determinante, este tipo de error adquiere una relevancia mucho mayor. Una respuesta aparentemente correcta, pero no respaldada por una fuente verificable, puede llevar a interpretaciones equivocadas, decisiones inadecuadas o pérdida de confianza en el

sistema. Esta situación resulta especialmente problemática en sectores regulados, donde la información debe ajustarse con exactitud a procedimientos, requisitos o normativas vigentes. En este sentido, el proyecto se plantea como una respuesta técnica a una limitación real de los modelos generativos: su incapacidad para garantizar, por sí solos, que cada afirmación emitida esté alineada con una fuente concreta y actualizada. (Huang, 2023)

La problemática adquiere especial relevancia en contextos donde la exactitud de la respuesta tiene implicaciones operativas o regulatorias. En el ámbito agrícola, la consulta de documentación técnica, boletines, instrucciones de aplicación o normativa asociada exige respuestas que no solo sean comprensibles, sino también correctas y justificables. En este sentido, la arquitectura RAG se plantea como una solución adecuada porque permite recuperar fragmentos específicos de un corpus documental antes de generar la respuesta, conectando la capacidad lingüística del modelo con una base de conocimiento externa y actualizable. (Li, 2025)

A partir de esta problemática, la arquitectura de Generación Aumentada por Recuperación se presenta como una solución especialmente adecuada. RAG permite que el modelo no responda exclusivamente a partir de su conocimiento paramétrico, sino que recupere primero fragmentos relevantes de un corpus documental y utilice esa evidencia para generar la respuesta final. Esto introduce una diferencia importante respecto al comportamiento de un LLM convencional: la respuesta deja de depender solo de patrones aprendidos durante el entrenamiento y pasa a apoyarse en información recuperada en tiempo de consulta. Diversos trabajos recientes destacan precisamente esta capacidad como uno de los mecanismos más efectivos para reducir alucinaciones y mejorar la fiabilidad en aplicaciones reales. (Li, 2025)

Desde la perspectiva de este TFM, esta característica no es un detalle técnico secundario, sino el núcleo del problema a resolver. El interés principal no consiste simplemente en construir un asistente que “responda bien”, sino en desarrollar un sistema que pueda justificar sus respuestas mediante documentación concreta. Dicho de otro modo, la utilidad del sistema no depende solo de la fluidez del lenguaje generado, sino de su capacidad para enlazar cada afirmación con una fuente verificable. En un dominio como el agrícola y normativo, donde es frecuente consultar documentos extensos, boletines técnicos, requisitos de ayuda, referencias de cumplimiento y criterios administrativos, la trazabilidad se convierte en un requisito funcional y no en un añadido opcional.

La justificación académica del proyecto también es clara. La investigación en LLMs, RAG y mitigación de alucinaciones avanza rápidamente, pero sigue existiendo una brecha entre los resultados obtenidos en entornos experimentales y la utilidad real en contextos documentales específicos. Buena parte de la literatura se centra en benchmarks generales, pruebas sintéticas o evaluaciones abstractas de fidelidad. Sin embargo, menos frecuentes son los estudios que implementan una solución completa sobre un corpus real, con restricciones de dominio, documentos heterogéneos y necesidad explícita de citación. Este TFM se sitúa

precisamente en ese espacio, aportando un caso de estudio aplicado en el que se diseña un sistema de recuperación y generación orientado a un problema concreto y evaluable. (Huang, 2023)

La motivación de este trabajo no parte únicamente de una inquietud teórica, sino también de una observación práctica. El crecimiento de los asistentes conversacionales ha extendido el uso de los LLMs a tareas de consulta, síntesis y apoyo a la decisión. Esta expansión ha sido positiva desde el punto de vista de la productividad, pero también ha puesto de manifiesto que no todos los dominios son igual de tolerantes al error. Cuando el conocimiento solicitado cambia con frecuencia, cuando existe un corpus documental extenso o cuando la consulta requiere interpretación de textos normativos, el uso de un LLM aislado resulta insuficiente. La necesidad de un mecanismo que conecte la generación con una base documental externa se vuelve, por tanto, evidente.

Además, el proyecto posee interés metodológico porque permite analizar de forma estructurada qué elementos contribuyen realmente a disminuir la probabilidad de error. No basta con afirmar que RAG mejora el comportamiento del modelo; es necesario demostrarlo mediante comparación con una línea base, métricas de recuperación, análisis de la calidad de la respuesta y estudio de la trazabilidad de las citas. Esta necesidad de evaluación convierte el trabajo en una propuesta rigurosa, ya que obliga a medir no solo la salida final del sistema, sino también los pasos intermedios que intervienen en la generación. En otras palabras, el proyecto no se limita a incorporar herramientas de IA, sino que evalúa su efecto sobre una tarea concreta. (Huang, 2023)

Desde un punto de vista tecnológico, la propuesta también resulta relevante porque los LLMs presentan una limitación estructural relacionada con la actualización del conocimiento. Un modelo entrenado sobre grandes volúmenes de datos no incorpora automáticamente los cambios más recientes que se producen en documentos normativos, boletines o instrucciones sectoriales. Esto es especialmente importante en el ámbito agrícola, donde la información puede variar por campañas, convocatorias, zonas geográficas o actualizaciones administrativas. En estas circunstancias, la actualización manual del conocimiento del modelo mediante reentrenamiento o ajuste fino resultaría costosa, lenta y poco flexible. RAG ofrece una alternativa más eficiente porque desacopla el conocimiento del modelo de la base documental externa, permitiendo actualizar el corpus sin modificar el modelo generativo.

1.2. Contexto del Proyecto

El presente trabajo se sitúa en el ámbito de la asistencia documental para el sector agrícola, donde la consulta de información normativa y técnica suele realizarse sobre documentos extensos, heterogéneos y con una estructura compleja. En este contexto, los usuarios no solo necesitan acceder a la información, sino también identificar rápidamente

la fuente exacta de cada dato y comprobar su vigencia. Esta necesidad se intensifica cuando los documentos contienen referencias a ayudas, requisitos de cumplimiento, fechas, condiciones técnicas o criterios administrativos que pueden cambiar con frecuencia. (Chen, 1999)

La evaluación de los modelos de lenguaje ha abierto nuevas posibilidades para automatizar este tipo de consultas, pero también ha puesto de manifiesto una limitación estructural: el conocimiento de estos modelos no se actualiza de forma continua y no siempre está alineado con documentos internos o normativos específicos. Los estudios recientes sobre alucinaciones muestran que este problema no es anecdótico, sino una barrera central para el uso fiable de los LLMs en dominios donde la precisión factual es crítica. (Huang, 2023)

Añadir, que la arquitectura RAG surge precisamente para abordar esta limitación. En lugar de depender únicamente del conocimiento paramétrico del modelo, el sistema recupera información de una base documental externa y la incorpora al proceso de generación. Esta separación entre generación y conocimiento permite mejorar la trazabilidad, actualizar el contenido sin reentrenar el modelo y limitar la respuesta a evidencia recuperada. La literatura reciente también destaca que RAG, combinado con estrategias de razonamiento y recuperación bien diseñadas, es una de las líneas consolidadas para mitigar las alucinaciones en aplicaciones reales. (Li, 2025)

1.3. Presentación del proyecto y del problema

El problema que aborda este trabajo consiste en que un modelo de lenguaje, utilizado de forma aislada, puede generar respuestas lingüísticamente correctas, pero no necesariamente verificables cuando se le consulta sobre documentación agrícola y normativa. Esta situación puede derivar en errores de interpretación, respuestas incompletas o afirmaciones que no se corresponden con el contenido real de los documentos. En un contexto regulado este comportamiento reduce la utilidad práctica del sistema y limita su adopción.

Asimismo, la propuesta desarrollada en este TFM consiste en implementar un sistema RAG local capaz de consultar un corpus documental previamente preparado, recuperar fragmentos relevantes y utilizar esa evidencia para generar respuestas más fiables. El sistema se orienta a mejorar tres aspectos principales: la reducción de alucinaciones, la trazabilidad de las respuestas y la utilidad operativa para usuarios que necesitan consultar documentación técnica de manera rápida y estructurada. (Zhong, 2025)

Para ello, el trabajo aborda el problema desde una perspectiva de ingeniería aplicada. En primer lugar, se prepara y fragmenta el corpus documental. Posteriormente, se construye una base vectorial para la recuperación semántica de información. A continuación, se integra un modelo de lenguaje para la generación de respuestas y se añade un mecanismo de citación y verificación. Finalmente, se evalúa el comportamiento del sistema mediante un conjunto de preguntas de prueba y métricas de recuperación y calidad de respuesta.

1.4. Utilidad real de la propuesta

La utilidad de este proyecto se fundamenta en que propone una forma concreta de mejorar la interacción y documentación técnica. En lugar de buscar manualmente entre múltiples archivos, el sistema permite formular consultas en lenguaje natural y obtener una respuesta acompañada de fragmentos relevantes del corpus. Esto reduce el tiempo de búsqueda y facilita la localización de información exacta dentro de documentos extensos.

Además, la incorporación de RAG aporta una ventaja clave frente a un LLM sin recuperación: la respuesta no depende únicamente del conocimiento aprendido durante el entrenamiento, sino que se apoya en información documental actualizada y controlada. En entorno como el agrícola, esta característica resulta especialmente valiosa porque la normativa y los criterios de aplicación puede modificarse con relativa frecuencia. La posibilidad de citar la fuente también refuerza la confianza del usuario y permite auditar la respuesta obtenida. (Li, 2025)

Desde el punto de vista técnico, el proyecto también tiene utilidad como caso de estudio sobre diseño y evaluación de sistemas RAG. Permite observar cómo influye la fragmentación del texto, la recuperación vectorial, el reranking y la formulación de consultas en la calidad de las respuestas. De este modo, el trabajo no se limita a implementar una solución funcional, sino que examina qué componentes mejoran realmente el comportamiento del sistema.

1.5. Estructura del trabajo

El resto del trabajo se organiza en capítulos que siguen una secuencia lógica de presentación, diseño, implementación y evaluación. El capítulo 2 recoge los objetivos general y específicos que guían la investigación, junto con su desglose en épicas, historias de usuario y sprints según Scrum. El capítulo 3 desarrolla el marco teórico y el estado del arte, centrado en los LLMs, la arquitectura RAG, la recuperación semántica y la problemática de las alucinaciones en entornos regulados. El capítulo 4 presenta el marco metodológico, donde se define el corpus documental, el diseño experimental y las variables de evaluación.

A continuación, el Capítulo 5 expone la implementación de la solución, incluyendo la arquitectura del sistema, el proceso de ingesta documental la indexación vectorial, la recuperación de contexto y el mecanismo de citación. Después, el capítulo 6 presenta los resultados obtenidos y su análisis comparativo. Finalmente, el capítulo 7 recoge las conclusiones, las limitaciones del trabajo y las posibles líneas futuras de desarrollo.

1.6. Estructura del trabajo

La memoria se organiza del siguiente modo:

1. Introducción, problema, contexto y utilidad.
2. Objetivos general y específicos, y planificación Scrum.
3. Marco teórico / estado del arte.
4. Marco metodológico (dataset, variables, diseño experimental).
5. Arquitectura e implementación del prototipo.
6. Despliegue operativo del prototipo funcional contenedorizado.
7. Evaluación, resultados y discusión (con limitaciones de evidencia).
8. Conclusiones, limitaciones y prospectiva.

Capítulo 2

Objetivos y planificación

El presente capítulo tiene como finalidad establecer de forma clara y precisa los objetivos que guían el desarrollo del Trabajo de Fin de Máster. En primer lugar se define el objetivo general, que representa la meta principal que se pretende alcanzar con la realización del proyecto. En segundo lugar se desglosan los objetivos específicos, que constituyen las partes en las que se subdivide el trabajo para completar el objetivo general. Cada objetivo se ha redactado en infinitivo y se ha diseñado para ser medible y alcanzable al finalizar el proyecto. Finalmente, cada objetivo específico se trata como una épica Scrum y se descompone en una lista de tareas ejecutables sobre el prototipo real: FastAPI con arquitectura hexagonal, PostgreSQL/pgvector, Ollama (Llama 3.2), React/Vite y Tailwind CSS.

2.1. Objetivo general

El objetivo general de este Trabajo de Fin de Máster es:

- Diseñar, implementar y evaluar un asistente inteligente para el sector agrícola basado en una arquitectura RAG agéntica que mejore la precisión, la relevancia y la trazabilidad de las respuestas generadas por modelos de lenguaje en un contexto normativo y técnico regulado. (Li, 2025)

Este objetivo engloba la finalidad última del proyecto: desarrollar un sistema capaz de resolver dudas técnicas y normativas de alta relevancia —como el cálculo de subvenciones según superficie de cultivo, los protocolos fitosanitarios en zonas vulnerables o la cumplimentación del Cuaderno Digital de Explotación Agrícola—, reduciendo la carga de trabajo de técnicos y asesores y mejorando la fiabilidad de las respuestas proporcionadas.

2.2. Objetivos específicos

Para alcanzar el objetivo general, el trabajo se ha subdividido en los siguientes objetivos específicos, cada uno de los cuales representa una épica que contribuye al cumplimiento del objetivo final:

- OE1.** Analizar el estado del arte en arquitecturas RAG, sistemas agénticos y aplicaciones de inteligencia artificial en el sector agrario, identificando las limitaciones de las soluciones existentes y las oportunidades de mejora. Este objetivo se corresponde con el marco teórico y el estado del arte desarrollado en el Capítulo 3.
- OE2.** Diseñar la arquitectura del sistema, definiendo los componentes de ingestación de datos, recuperación semántica, orquestación agéntica, generación de respuestas y presentación de la información. Este objetivo se desarrolla en el Capítulo 5, donde se describen las decisiones de arquitectura y los componentes del sistema.
- OE3.** Implementar el prototipo funcional, integrando la base de datos vectorial (PostgreSQL/pgvector), el modelo de lenguaje (Llama 3.2 a través de Ollama), el núcleo RAG en FastAPI con arquitectura hexagonal y la interfaz de usuario desarrollada con React, Vite y Tailwind CSS. Este objetivo se aborda en los Capítulos 5 y 6.
- OE4.** Desarrollar el módulo de trazabilidad y citación automática, garantizando que cada respuesta incluya la referencia exacta al documento y la página de donde se extrajo la información. Este módulo es fundamental para reducir las alucinaciones y facilitar la verificación por parte del técnico.
- OE5.** Evaluar el rendimiento del sistema en términos de fidelidad, relevancia, latencia y uso de recursos, utilizando el framework RAGAS y pruebas en hardware local, así como analizar la experiencia de usuario y las barreras de adopción. Este objetivo se cumple en el Capítulo 7.
- OE6.** Documentar el proceso de desarrollo, las decisiones de diseño y los resultados obtenidos, proporcionando un prototipo funcional y una guía de implementación que pueda servir como referencia para futuros trabajos en el ámbito de la inteligencia artificial aplicada al sector agrario. Este objetivo se refleja en la estructura completa del TFM y en los anexos del documento.

2.3. Planificación Scrum y lista de tareas

Cada objetivo específico se gestiona como una épica Scrum (Schwaber y Sutherland, 2020). El equipo es unipersonal: el autor prioriza el backlog, implementa el incremento

y presenta la revisión al tutor. El trabajo se reduce a siete tareas, redactadas en infinitivo y ordenadas por dependencia: investigación, diseño, backend, frontend, trazabilidad, evaluación y cierre documental.

2.3.1. Tarea 1. Documentación, investigación y análisis del estado del arte

Épica OE1.

- Revisar la literatura sobre LLMs, alucinaciones, RAG y sistemas agénticos.
- Comparar enfoques de orquestación (LangChain, LlamaIndex, Haystack e implementación hexagonal propia) y justificar la elección del prototipo.
- Identificar limitaciones de los LLM autónomos en normativa agrícola (PAC, POSEI, sanidad vegetal).
- Extraer requisitos de diseño: anclaje documental, citación, evaluación desacoplada y despliegue local.
- Redactar el Capítulo 3.

2.3.2. Tarea 2. Diseño del prototipo, las entidades y el dominio

Épica OE2.

- Definir los flujos de usuario (login, chat, datasets, organización, evaluación) y el stack del MVP: FastAPI, PostgreSQL/pgvector, Ollama (Llama 3.2), React/Vite y Tailwind CSS.
- Diseñar la arquitectura lógica: ingesta, recuperación híbrida, orquestación agéntica, generación y presentación.
- Delimitar el *bounded context* RAG y elaborar el diagrama hexagonal de puertos y adaptadores (Cockburn, 2005).
- Modelar las entidades de dominio (`RetrievedChunk`, `RetrievalQuery`, `RetrievalBundle`) y los puertos de salida, sin dependencia de FastAPI ni SQLAlchemy.
- Especificar la fusión RRF, la expansión léxica agrícola, el aislamiento por `tenant_id` y los metadatos de documento, sección y página.

2.3.3. Tarea 3. Diseño y creación del back-end

Épica OE3.

- Implementar la capa de dominio en `gateway/rag/domain`.
- Crear los casos de uso: `AnswerQuestion`, `HybridRetrieve`, `IngestDocument`, `EvaluateRetrieval` y `EvaluateAnswer`.
- Crear los adaptadores de salida: `PostgreSQL/pgvector`, `BM25`, `Ollama` y `Cross-Encoder`.
- Crear los controladores FastAPI (`chat`, `datasets`, `evaluación`, `validación humana`) y la fachada `RAGService`.
- Implementar el modo agéntico (`AgenticRAGService`) y el composition root `build_rag_container`.

2.3.4. Tarea 4. Diseño y creación del front-end

Épica OE3.

- Implementar autenticación y vista principal del panel (`organización`, `cuenta` y `navegación`).
- Crear la vista del asistente conversacional, con modo híbrido o agéntico y visualización de fuentes.
- Crear la vista de `datasets`: `alta de corpus`, `previsualización de fragmentos` e `indexación`.
- Crear el laboratorio de evaluación, el historial de experimentos y la validación humana.
- Incluir, como módulo satélite, el cuaderno de campo.

2.3.5. Tarea 5. Módulo de trazabilidad y citación automática

Épica OE4.

- Enriquecer cada fragmento en la ingesta con `documento`, `sección` y `página de origen`.
- Propagar las citas desde la recuperación hasta la respuesta y la interfaz de chat.
- Registrar trases (`pregunta`, `contexto`, `respuesta` y `fuentes`) para auditoría.
- Aplicar guardrails de `ámbito`, `no invención`, `cifras no soportadas` y `detección de PII`.
- Garantizar que, si no hay evidencia recuperada, el sistema declare la carencia en lugar de alucinar.

2.3.6. Tarea 6. Evaluación del rendimiento y de la adopción

Épica OE5.

- Construir el banco de pruebas del dominio agrario y el laboratorio de estrategias de recuperación.
- Medir fidelidad, relevancia, precisión y exhaustividad de contexto con RAGAS y LLM-as-a-judge.
- Comparar el LLM sin RAG frente al prototipo con RAG en las mismas preguntas.
- Medir latencia y uso de recursos en hardware local.
- Analizar la experiencia de uso y las barreras de adopción mediante validación humana.

2.3.7. Tarea 7. Redacción de la memoria y presentación final

Épica OE6.

- Redactar los capítulos de la memoria, la guía de arranque con Docker Compose, el glosario y la bibliografía.
- Preparar la defensa: problema, arquitectura hexagonal, demostración del asistente y resultados de evaluación.
- Seleccionar casos de consulta normativa con citas verificables.
- Exponer limitaciones (consultas holísticas, OCR, alcance textual de la evaluación) y líneas futuras.

2.4. Correspondencia entre objetivos y tareas

La tabla 2.1 hace explícito que cada épica queda cubierta por al menos una tarea, y que el objetivo general solo se considera alcanzado cuando las siete tareas están cerradas.

Tabla 2.1: Correspondencia entre objetivos específicos (épicas) y las siete tareas del MVP.

Épica	Tarea	Entregable de cierre
OE1	Tarea 1	Capítulo 3
OE2	Tarea 2	Arquitectura, entidades y dominio hexagonal
OE3	Tareas 3 y 4	Prototipo backend + frontend contenedorizado
OE4	Tarea 5	Citas por documento y página; trases auditables
OE5	Tarea 6	Capítulo 7 (RAGAS, latencia, UX)
OE6	Tarea 7	Memoria, anexos y defensa

Quedan fuera de alcance de esta edición el razonamiento visual end-to-end, la certificación industrial del despliegue y el reentrenamiento o ajuste fino del LLM generativo. El cuaderno de campo se contempla como módulo satélite de la Tarea 4, no como condición para dar por cumplido el objetivo general.

Capítulo 3

Marco teórico / Estado del arte

3.1. Arquitectura Transformers y Grandes Modelos del Lenguaje

Los grandes modelos de lenguaje (LLMs, por sus siglas en inglés) constituyen una de las principales líneas de avance en el Procesamiento del Lenguaje Natural (PLN) debido a su capacidad para modelar dependencias complejas entre tokens y generar texto coherente en múltiples tareas de comprensión y razonamiento. Su desarrollo contemporáneo se fundamenta en la arquitectura Transformer, introducida por Vaswani et al. (2017), la cual desplazó a los modelos secuenciales previos al proponer un mecanismo de atención capaz de representar relaciones contextuales de forma altamente paralelizables.

A diferencia de las Redes Neuronales Recurrentes (RNN) o las redes de Memoria a Corto y Largo Plazo (LSTM), que procesan la información de manera estrictamente secuencial y sufren de degradación de la señal en secuencias largas (el problema del desvanecimiento del gradiente), el Transformer permite ponderar simultáneamente la interacción entre todos los elementos de una secuencia. Esta arquitectura asume un rol fundacional en este trabajo, ya que constituye el motor inferencial generativo del sistema; sin embargo, comprender sus fundamentos matemáticos es indispensable para identificar por qué, estructuralmente, estos modelos son propensos a la alucinación factual en corpus normativos como la Política Agraria Común (PAC).

Tabla 3.1: Comparación de arquitecturas y enfoques NLP relevantes para este trabajo.

Arquitectura	Ventaja	Limitación
RNN	Memoria secuencial	Desvanecimiento del gradiente
LSTM	Dependencias más largas	Lentitud y coste de entrenamiento
Transformers	Paralelismo y contexto amplio	Alto coste computacional
RAG	Conocimiento actualizable y citable	Dependencia de la recuperación

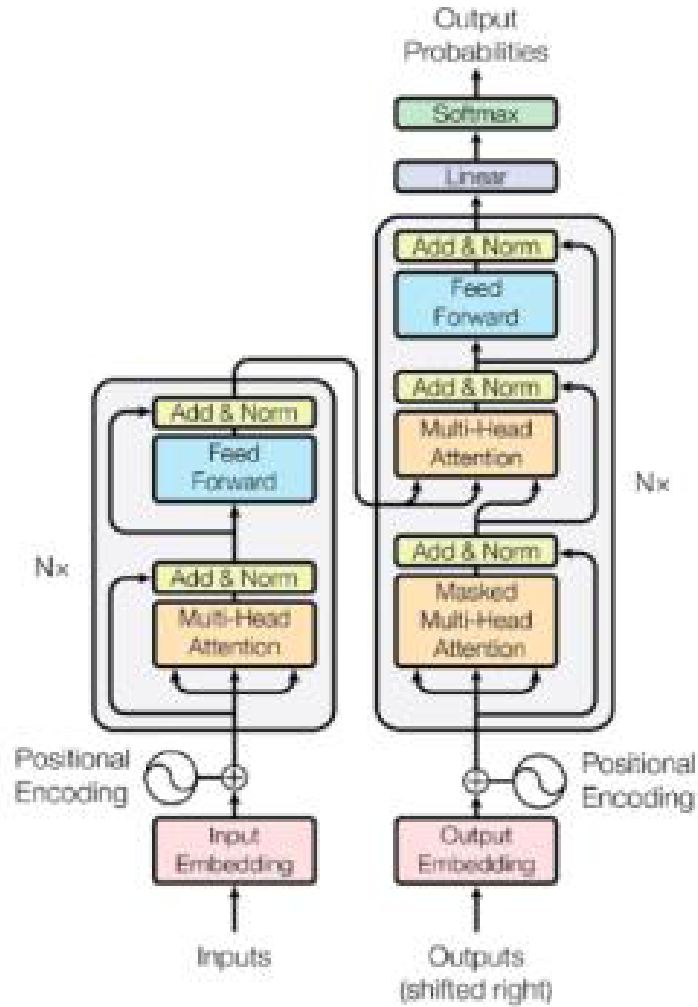


Figura 3.1: Arquitectura Transformer (codificador–decodificador) con atención multi-cabeza, conexiones residuales y normalización (Vaswani et al., 2017).

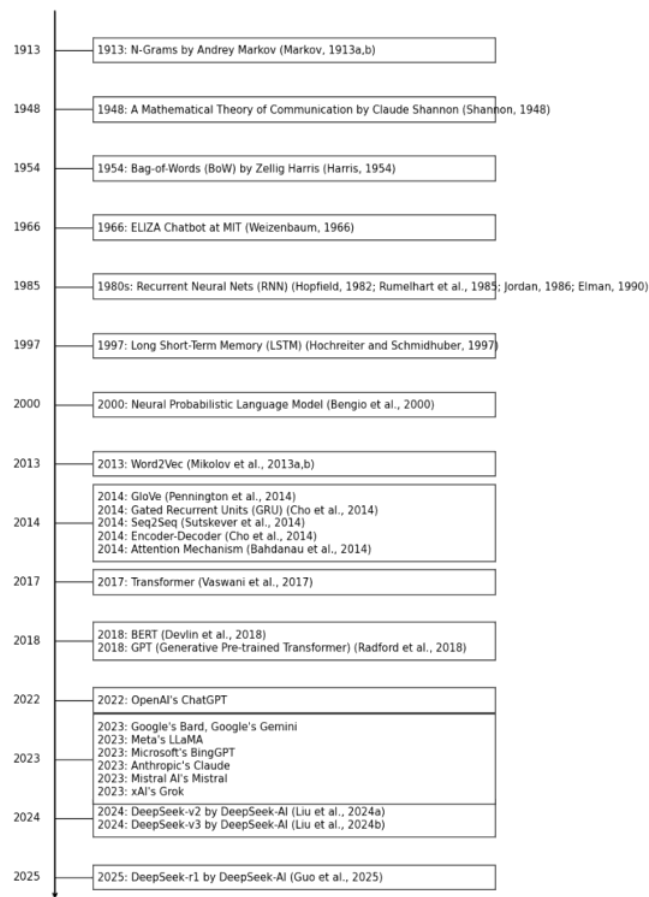


Figura 3.2: Línea temporal de modelos de lenguaje, desde n-gramas y RNN/LSTM hasta Transformer, BERT, GPT y los LLM contemporáneos.

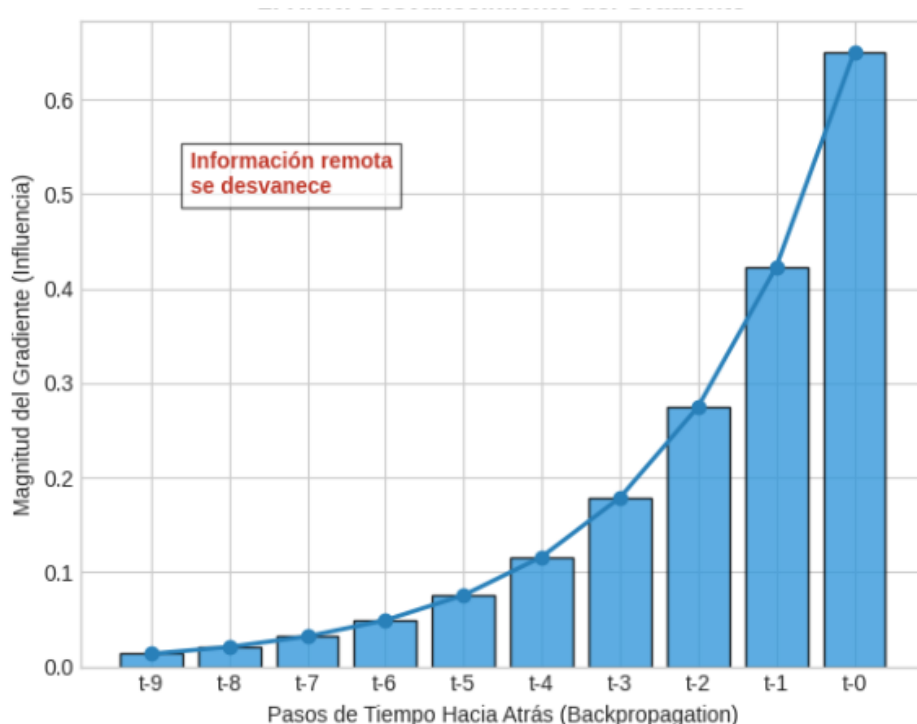


Figura 3.3: Desvanecimiento del gradiente en retropropagación: la magnitud de la señal decae al alejarse del instante actual, limitando a las RNN clásicas.

3.1.1. Fundamentos Matemáticos de los Transformers y Mecanismos de Atención

Desde el punto de vista arquitectónico, el núcleo de un Transformer prescinde de la recurrencia y la convolución, basándose íntegramente en el mecanismo de Autoatención de Producto Escalar Escalado (Scaled Dot-Product Attention). La autoatención permite que cada token integre información procedente del resto de la secuencia, generando representaciones contextualizadas que dependen del entorno lingüístico dinámico y no solo de una incrustación léxica estática.

Mecanismo de Autoatención, el modelo proyecta linealmente las representaciones de los tokens en tres matrices distintas: Query (Q), Key (K) y Value (V). Estas proyecciones se obtienen multiplicando la matriz de entrada (que contiene los embeddings de los tokens) por matrices de pesos aprendibles W_Q , W_K y W_V :

La función de atención calcula la similitud entre cada Query y todas las Keys mediante un producto escalar. Para evitar que valores extremadamente grandes empujen la función softmax hacia regiones con gradientes infinitesimales (lo que estancaría el aprendizaje), el producto se escala dividiéndolo por la raíz cuadrada de la dimensión de las keys ($\sqrt{d_k}$). Finalmente, se aplica la función softmax para obtener los pesos de atención, los cuales multiplican a la matriz de valores (V):

Esta ecuación explica matemáticamente por qué los LLMs pueden producir respuestas

sintácticamente fluidas y semánticamente plausibles: cada palabra generada es una amalgama probabilística altamente contextualizada del texto anterior. Sin embargo, esta misma formulación evidencia su principal carencia normativa: la matriz se construye basándose en pesos internalizados durante el preentrenamiento. No existe en esta ecuación ningún mecanismo de verificación contra una base de datos externa que garantice la veracidad de la correlación estadística.

Durante la fase de inferencia de un modelo generativo (arquitecturas Decoder-only como la familia LLaMA o GPT, prevalentes en tareas de Q&A), el modelo opera de forma autorregresiva. Esto implica que la predicción del token en el instante temporal se condiciona a la probabilidad conjunta de todos los tokens generados hasta el instante :

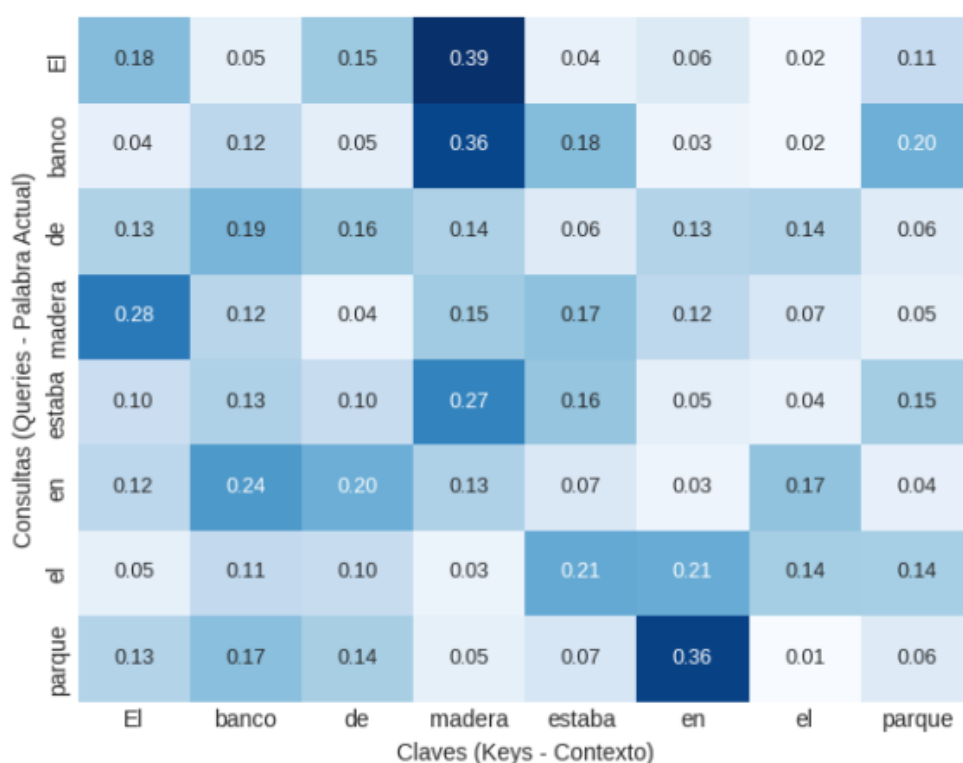


Figura 3.4: Matriz de autoatención sobre la oración «El banco de madera estaba en el parque»: cada consulta pondera las claves del contexto. Ilustra por qué el sentido de un token depende de su entorno y no de un embedding estático.

3.1.2. Limitaciones Estructurales de los LLMs

La aplicación directa de Modelos de Lenguaje de Gran Escala aislados (standalone) para la consulta y gestión de documentación técnica o regulatoria presenta severas inconsistencias funcionales y metodológicas. Aunque los LLMs exhiben una notable capacidad para comprender el lenguaje natural, resumir textos complejos y generar respuestas gramaticalmente fluidas, su arquitectura subyacente impone barreras insuperables cuando se exige rigor jurídico, actualización documental estricta y trazabilidad en el ámbito agrario

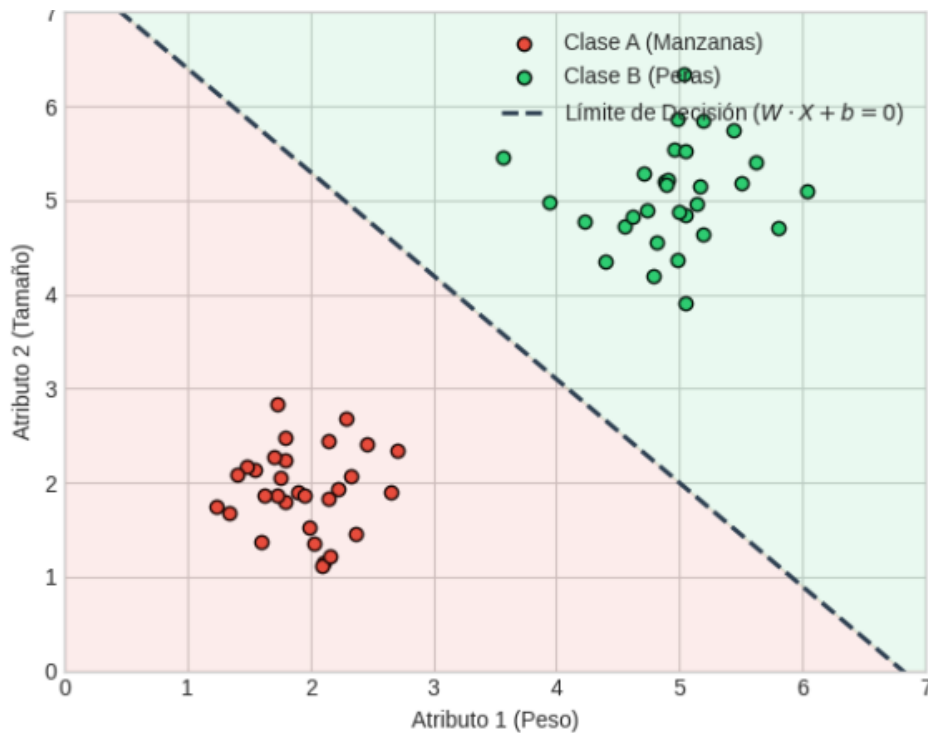


Figura 3.5: Hiperplano de decisión lineal $W \cdot X + b = 0$ en un espacio de dos atributos. Sirve de analogía geométrica a la separación de representaciones, previa a los espacios densos de los embeddings.

(por ejemplo, en la regulación de las ayudas de la Política Agraria Común —PAC— o el Programa de Opciones Específicas por la Lejanía y la Insularidad —POSEI—).

- Memoria paramétrica frente a Memoria Paramétrica. La diferencia fundamental entre un sistema de información tradicional y un LLM standalone reside en la naturaleza de su almacenamiento de conocimiento:
- Memoria Paramétrica: Todo el conocimiento de un LLM reside de forma distribuida e implícita en los pesos numéricos de sus capas de atención y redes feed-forward, ajustados durante la fase de preentrenamiento masivo. El modelo no "consulta" un documento en tiempo de inferencia; en su lugar, computa una distribución de probabilidad sobre el vocabulario

3.1.3. Taxonomía del Problema de las Alucinaciones en Dominios Normativos

Se define la alucinación en los LLMs como la generación de texto que es fácticamente incorrecto, desalineado del contexto o carente de sustento en la evidencia disponible (Huang et al., 2025; Ji et al., 2023). En el procesamiento de normativas agrícolas como la PAC o el POSEI, las alucinaciones se categorizan estructuralmente en dos tipos:

Alucinaciones Intrínsecas: La salida del modelo contradice explícitamente la información proporcionada en el contexto o en el prompt de entrada. Ocurre cuando el mecanismo de atención pondera erróneamente tokens de contexto contiguos, pero semánticamente antagónicos.

Alucinaciones Extrínsecas: El modelo genera afirmaciones que no pueden ser verificadas ni refutadas a partir del contexto provisto (o cuando se le consulta de forma directa sin contexto). El LLM fabrica datos, artículos de ley inexistentes o requisitos de elegibilidad plausiblemente redactados, pero jurídicamente nulos.

En el marco de la regulación agrícola, una alucinación extrínseca —como inventar una excepción normativa en el plazo de presentación de solicitudes de pago por ecorregímenes— conduce a la invalidación del sistema, ya que invalida el criterio de seguridad jurídica y puede derivar en sanciones económicas directas para los solicitantes.

3.1.4. Restricciones en la Ventana de Contexto y el Fenómeno Lost in the Middle

Un intento ingenuo de resolver la falta de conocimiento paramétrico en un LLM standalone consiste en inyectar el texto normativo completo (cientos de páginas del Boletín Oficial del Estado o reglamentos comunitarios de la UE) dentro de la ventana de contexto (context window) de un modelo con capacidad extendida (ej. 32k, 128k o más tokens). Sin embargo, esta estrategia colapsa por dos razones estructurales:

Complejidad Computacional: El mecanismo de autoatención escala de forma cuadrática con respecto a la longitud de la secuencia, elevando exponencialmente la latencia y la demanda de memoria VRAM.

Degradación del Posicionamiento Semántico (Lost in the Middle): Como demostraron Liu et al. (2023), los modelos basados en Transformer muestran una marcada asimetría en la capacidad de recuperación dentro del contexto de entrada. Los modelos son altamente eficaces identificando e integrando información situada al inicio (primacy effect) o al final (recency effect) del prompt, pero sufren un menoscabo drástico en el rendimiento cuando la información clave se halla inmersa en la zona central del texto inyectado. En normativas de gran volumen, los condicionantes técnicos clave suelen estar sepultados en anexos intermedios, quedando efectivamente “invisibilizados” para la atención del modelo.

3.1.5. Volatilidad Normativa y Falta de Determinismo y Trazabilidad

Finalmente, la gestión de la normativa agraria exige resolver tres factores críticos incompatibles con los LLMs aislados:

Volatilidad y Asincronía Temporal: Las leyes agrícolas están sujetas a modificaciones

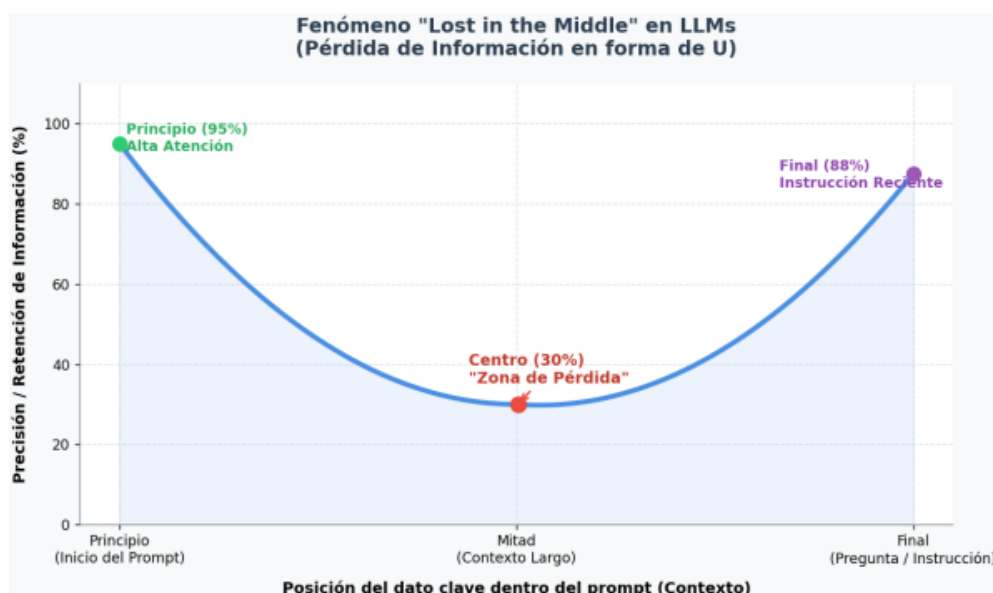


Figura 3.6: Fenómeno *Lost in the Middle*: la retención de un dato clave cae en el centro del prompt y se recupera en los extremos (primacía y recencia).

periódicas (órdenes de modificación de la PAC, resoluciones autonómicas de sequía, ajustes en las fichas del POSEI). Un LLM standalone es estático; su conocimiento termina de forma tajante en su fecha de corte de entrenamiento (knowledge cutoff). La única vía para actualizar la memoria paramétrica de un LLM es mediante el reentrenamiento o el Ajuste Fino (Fine-Tuning), procesos prohibitivos en términos de tiempo, costes computacionales y riesgo de catastrophic forgetting (olvido catastrófico de habilidades previas).

Ausencia de Trazabilidad y Citación Granular: Un LLM aislado opera como una "caja negra" probabilística. Al generar una respuesta sobre un requisito técnico, no es capaz de proporcionar la dirección exacta de la fuente (p. ej., Reglamento (UE) 2021/2115, Artículo 14, apartado 2, página 45). La cita producida por un LLM aislado suele ser una alucinación estilística basada en patrones de citación aprendidos.

Falta de Determinismo: Dado que el proceso de muestreo en la capa de salida (decoding) emplea parámetros como la temperatura () o el muestreo top-, la misma consulta formulada en momentos distintos puede derivar en interpretaciones contradictorias sobre un mismo marco regulatorio.

3.2. Arquitectura Retrieval-Augmented Generation (RAG)

La arquitectura Retrieval-Augmented Generation (RAG), introducida formalmente por Lewis et al. (2020), nace como un paradigma híbrido que desacopla el almacenamiento del conocimiento de la capacidad de razonamiento del modelo de lenguaje. En lugar de confiar exclusivamente en la memoria paramétrica interna del LLM, RAG integra un componente

de recuperación semántica externa (Memoria Paramétrica) que extrae dinámicamente evidencia documental relevante desde una base de conocimiento en tiempo de inferencia, inyectándola directamente en el contexto de entrada del generador.

En el ámbito de la documentación normativa agraria (PAC/POSEI), esta arquitectura no se concibe como un mero sustituto del LLM, sino como un mecanismo de anclaje estricto (Strict Grounding) que condiciona la probabilidad de generación del modelo a la evidencia recuperada, mitigando estructuralmente las alucinaciones extrínsecas y posibilitando la trazabilidad exacta de las respuestas.

3.2.1. Descomposición del Pipeline de RAG: Indexación, Recuperación y Generación

El funcionamiento de un sistema RAG en entornos regulados se divide operativamente en dos fases principales: Indexación Asíncrona (Ingesta) y Pipeline de Inferencia Síncrona (Recuperación y Generación).

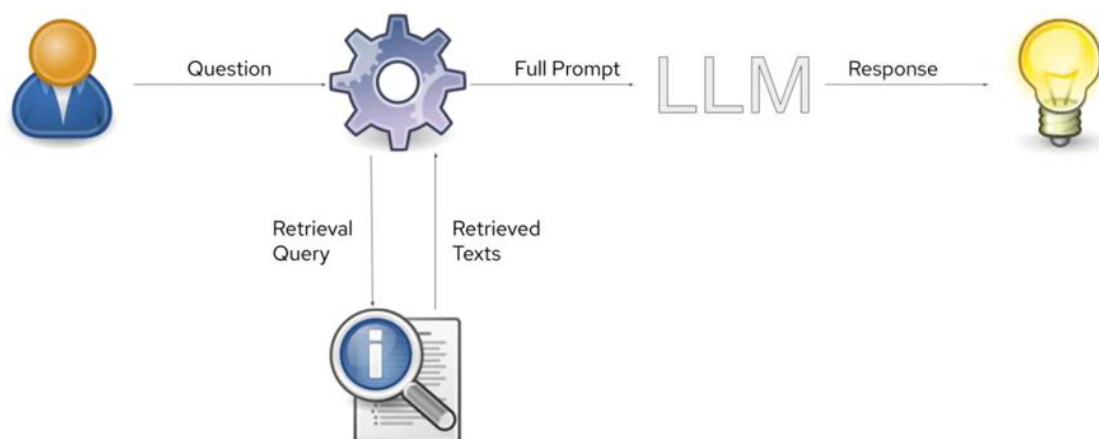


Figura 3.7: Pipeline RAG: la pregunta dispara una consulta de recuperación; los textos recuperados se concatenan al prompt completo y el LLM genera la respuesta anclada.

En este trabajo, la arquitectura RAG no se interpreta como una solución automática al problema de las alucinaciones, sino como un mecanismo cuya eficacia debe ser evaluada empíricamente. La literatura reciente sobre RAGAS distingue explícitamente entre métricas de recuperación y métricas de generación, como context precision, context recall, faithfulness y answer relevancy, precisamente porque la utilidad de un sistema RAG no puede inferirse solo a partir de la fluidez de la respuesta. Por ello, el presente estudio adopta una visión cuantitativa en la que la recuperación semántica y la generación se analizan por separado y en conjunto.

Fase de Indexación y Preparación Documental:

Parseo y Extracción de Maquetación (Layout Analysis): Los documentos normativos en formato PDF se procesan mediante herramientas avanzadas (e.g., Docling con soporte

OCR) que preservan la estructura jerárquica, tablas y elementos visuales, exportándolos a sintaxis Markdown estructurada.

Fragmentación Semántica (Chunking): El texto unificado se segmenta en bloques discretos. Para evitar la rotura de contexto normativo en medio de un artículo o tabla, se aplican estrategias de fragmentación por encabezados o Summary-Augmented Chunking (Reuter et al., 2025).

Incrustación Vectorial (Embedding): Cada fragmento se pasa por un modelo codificador (Encoder) para proyectarlo en un espacio vectorial denso de dimensión :

Los vectores resultantes, junto con metadatos de gobernanza (tenant_id, página, sección, checksum), se almacenan en un índice espacial.

La calidad de esa proyección condiciona todo el retriever denso: si dos redacciones distintas de la misma norma (por ejemplo, “ayuda al arranque de viñedo” y “prima por abandono de viña”) no quedan próximas en el espacio, el generador nunca verá el artículo correcto. Por ello conviene distinguir las familias de embeddings que hoy dominan la recuperación semántica y justificar cuál se instanció en el prototipo.

Sentence Transformers. Sentence Transformers es una familia de modelos basados en BERT que generan embeddings a nivel de oración o párrafo (Reimers y Gurevych, 2019). Frente al BERT original, que produce representaciones de token y no estaba entrenado para comparar frases con similitud coseno, estos modelos se afinan con aprendizaje siamés o tripletas de modo que la distancia entre vectores refleje similitud semántica. Variantes como `all-MiniLM-L6-v2` (384 dimensiones, muy ligera) o `paraphrase-multilingual-mpnet-base-v2` (768 dimensiones, mejor cobertura idiomática) se usaron de forma masiva como primer retriever denso en prototipos RAG. Su ventaja es la eficiencia y el ecosistema Hugging Face; su límite, en un corpus normativo en español, es que los checkpoints más citados están optimizados para parafraseo general o para inglés, y degradan cuando la consulta es una paráfrasis agraria y el documento es un artículo del POSEI con identificadores alfanuméricos.

BGE (BAAI General Embeddings). BGE, desarrollada por la Academia de Inteligencia Artificial de Pekín, reentrenó el paradigma de embedding para búsqueda: instrucciones de consulta, normalización L2 y evaluación sistemática en MTEB (Xiao et al., 2023). El modelo de referencia para escenarios no anglófonos no es un hipotético `bge-large-es` —que no forma parte del catálogo oficial de BAAI— sino `BAAI/bge-m3`: 1024 dimensiones, más de 100 idiomas, ventanas de hasta 8192 tokens y capacidad de devolver a la vez vector denso, pesos dispersos y multi-vector tipo ColBERT (Chen et al., 2024). En este trabajo BGE no se descartó: se reservó para la segunda etapa del pipeline, el Cross-Encoder `BAAI/bge-reranker-v2-m3`, que reordena los candidatos ya fusionados por RRF. Así se aprovecha la fortaleza de BGE en ranking fino sin duplicar el coste de reindexar todo el corpus con su encoder bi-encoders.

E5 (Embeddings from bidirectional Encoder Representations). E5, de Mi-

Microsoft, introduce un entrenamiento débilmente supervisado y, sobre todo, una asimetría explícita entre consulta y documento (Wang et al., 2022). En inferencia se prefiere `query:` o `passage:`, de modo que el espacio vectorial no trata la pregunta del inspector y el párrafo del boletín como el mismo tipo de objeto. `intfloat/multilingual-e5-large` (1024 dimensiones) ofrece un compromiso sólido en español y en recuperación asimétrica. El inconveniente operativo para este prototipo es que E5 vive de forma natural en Hugging Face Transformers, no en el runtime Ollama con el que se sirve el generador local; adoptar E5 habría partido la inferencia en dos stacks (Transformers para embedder y Ollama para LLM).

Qwen3 Embedding (elección del prototipo). La familia Qwen3 Embedding (Zhang et al., 2025) hereda el backbone Qwen3 y publica tres tamaños (0.6B, 4B y 8B) pensados para recuperación, no solo para similitud de frases. El tag `qwen3-embedding:latest` servido por Ollama corresponde a la variante 8B: más de 100 idiomas, contexto de 32k tokens, vectores de hasta 4096 dimensiones con representación tipo Matryoshka e *instruction-aware* (la consulta puede llevar una instrucción de tarea, análogo funcional a E5). En junio de 2025 el modelo 8B encabezaba el MTEB multilingüe (70,58). En AgroPS ese checkpoint es el embedder por defecto (`RAG_EMBEDDING_MODEL`): se persiste en pgvector, se recorta a 2000 dimensiones para el índice HNSW y convive con un catálogo de contraste (`bge-m3`, `nomic-embed-text`, `mxbai-embed-large`) para experimentos embedding $\times$ distancia.

Fase de Inferencia: Recuperación Semántica (Retriever):

Al recibir una consulta del usuario (ej. "¿Cuáles son los requisitos de elegibilidad para la ayuda al arranque de viñedo en el POSEI?"), el sistema la proyecta al mismo espacio latente:

El recuperador evalúa la similitud entre la consulta y los fragmentos indexados. La métrica habitual en espacios normativos es la similitud coseno:

Búsqueda Híbrida y Re-Ranking: Dado que la búsqueda densa por similitud coseno puede fallar al buscar identificadores alfanuméricos exactos o códigos de ley (ej. "Orden APA/102/2024"), se implementa un esquema de Búsqueda Híbrida que combina la similitud semántica con un algoritmo léxico clásico (). Los top- fragmentos devueltos por ambos métodos se fusionan y reordenan mediante un modelo Cross-Encoder (Re-ranker) para maximizar la precisión del contexto inyectado.

Fase de Generación Condicionada (Generator):

Los fragmentos con mayor relevancia semántica se ensamblan en un modelo de plantilla de prompt (Prompt Template) junto con la consulta original y una instrucción estricta de sistema (system prompt).

Tabla 3 Tipos de RAG

Tipo

Ventajas

Tabla 3.2: Justificación de `qwen3-embedding:latest` frente a Sentence Transformers, BGE y E5, y efecto esperado sobre la recuperación semántica.

Criterio	Por qué se eligió este modelo	Impacto en la recuperación semántica
Idioma del corpus	Español técnico PAC/POSEI, no inglés de MTEB clásico. Qwen3 cubre >100 idiomas; MiniLM y <code>bge-large-en</code> están sesgados al inglés; no existe <code>bge-large-es</code> oficial.	Acerca paráfrasis del agricultor o del gestor (“prima de arranque”) al tenor literal del boletín, elevando <code>recall@k</code> del retriever denso.
Runtime del prototipo	Inferencia local unificada en Ollama, junto al LLM. E5 y Sentence Transformers exigirían un stack Hugging Face paralelo.	Misma GPU/CPU, misma API de embeddings y reindexado reproducible sin servicios cloud.
Asimetría consulta/documento	Qwen3 es <i>instruction-aware</i> ; E5 también, pero BGE-M3 ya no exige prefijos y MiniLM trata query y passage igual.	Mejora el ranking cuando la pregunta es corta y el chunk es un artículo largo, típico de la consulta normativa.
Dimensionalidad y MRL	Hasta 4096 dimensiones (se indexan 2000 en HNSW). MiniLM usa 384; E5/BGE-M3, 1024.	Espacio más expresivo para matices (excepciones, anexos, ecorregímenes) sin reentrenar; el recorte Matryoshka conserva la geometría semántica.
Ventana de contexto	32k tokens frente a 512 de MiniLM/BGE-en o 8192 de BGE-M3. Los chunks del prototipo son de 1000 caracteres.	El embedding del fragmento captura el artículo completo, no un recorte a 512 tokens que parte tablas LMR o listas PAC.
Papel de BGE en el sistema	BGE no se usa como embedder por defecto; se usa como Cross-Encoder <code>bge-reranker-v2-m3</code> sobre los candidatos RRF.	La recuperación semántica gana en dos tiempos: Qwen3 amplía el <i>recall</i> denso; BGE reordena precisión en el top- <i>k</i> final.
Complemento léxico	El denso no sustituye a BM25: los códigos (“Orden APA/102/2024”) siguen yendo al índice léxico.	Menos document-level mismatch: sinonimia (denso) + identificador exacto (BM25) + RRF.

Casos de Uso
 Naïve
 Simple
 Chatbots
 Graph
 Relaciones
 Jurídico
 Agentic Rag
 Planificación
 Investigación
 Modular RAG
 Empresas
 Producción

3.3. Revisión Crítica de la Arquitectura RAG en Dominios Regulados

En los últimos años han proliferado estudios sobre arquitecturas Retrieval-Augmented Generation (RAG) para mitigar alucinaciones en LLMs, especialmente en dominios regulados como la medicina y el derecho. Estas investigaciones coinciden en que RAG incorpora conocimiento externo “no paramétrico” (por ejemplo, bases de datos vectoriales o textos normativos) al proceso de generación, lo que reduce la tasa de respuestas erróneas o fabricadas. Por ejemplo, en entornos médicos especializados, Wada et al. (2025) demostraron que un modelo local de radiología potenciado con RAG eliminó por completo las alucinaciones en consultas sobre medios de contraste (0 % frente a 8 % con el modelo base; $p=0.012$). De forma similar, He et al. (2025) mostraron que un modelo GPT-4 con RAG alcanzó un 96.4 % de precisión en evaluaciones preoperatorias (vs. 86.6 % humano) sin generar alucinaciones. En el ámbito jurídico, Schwarcz et al. (2025) hallaron que una herramienta jurídica basada en RAG mejoraba la calidad del trabajo (claridad y profesionalismo) y “redujo las alucinaciones a niveles comparables” al trabajo humano sin IA. Igualmente, Rabnam et al. (2025) desarrollaron el framework LQ-RAG para consultas legales, logrando mejoras significativas en métricas de recuperación (por ejemplo, +13 % de hit rate y +15 % de MRR) mediante agentes de auditoría y modelos afinados, aunque sin incorporar aún retroalimentación humana.

En agricultura, Sawant et al. (2026) diseñaron un sistema RAG para asesoría agronómica, procesando documentos de prácticas agrarias mediante chunking semántico y embeddings especializados. Evaluaron varios LLMs (Llama3.1, Mistral, Phi3, Qwen2.5) utilizando métricas de relevancia y recuperación (precisión@K, recall@K, MRR, NDCG) y

métricas de calidad semántica (BLEU, ROUGE, BERT, Score, faithfulness). Sus hallazgos indican que modelos como Mistral y Qwen2.5 obtienen mejor desempeño en relevancia y calidad de respuesta en el dominio agrícola. No obstante, casi ninguna investigación previa ha aplicado RAG explícitamente a la documentación normativa agraria (PAC/POSEI), lo que señala una brecha importante.

3.4. Alucinaciones en modelos de lenguaje en entornos normativos (definición, causas y estudios relevantes).

Los enfoques de Retrieval-Augmented Generation (RAG) combinan la recuperación de información externa con la generación de texto para enriquecer las respuestas de los LLM con evidencias verificables. La bibliografía resalta que RAG pretende mitigar las limitaciones paramétricas de los LLM, mejorando la calidad factual de las respuestas y reduciendo la incidencia de alucinaciones o afirmaciones no fundamentadas. En este contexto, una alucinación se entiende como una salida factualmente incorrecta que el modelo genera de manera plausible pero no está respaldada por la información recuperada. Sin embargo, varios autores advierten que la eficacia de RAG depende críticamente de la precisión de la etapa de recuperación: por ejemplo, Reuter et al. (2025) señalan que RAG sólo reduce las alucinaciones cuando la recuperación documental es fiable, advirtiendo fallos como el Document-Level Retrieval Mismatch donde se seleccionan textos irrelevantes. En suma, RAG no elimina automáticamente las alucinaciones; su beneficio debe verificarse empíricamente caso a caso.

La revisión de trabajos previos muestra diversas arquitecturas RAG especializadas en dominios regulados. Xu et al. (2025) introducen MEGA-RAG, un framework RAG para salud pública con múltiples fuentes de evidencia. Reportaron una reducción de alucinaciones de más del 40 % frente al modelo base y alta exactitud en las respuestas (precisión $\approx 0,79$). En el ámbito legal, Rahman et al. (2025) proponen LQ-RAG, que integra agentes de auditoría y modelos finamente ajustados para consultas jurídicas. Su evaluación muestra mejoras relevantes en métricas de recuperación (por ejemplo, +13 % en Hit Rate, +15 % en MRR) y en la relevancia de las respuestas (hasta +23 % vs configuración ingenua). Feng et al. (2026) presentan Hyper-RAG, que emplea modelos de conocimiento basados en hipergrafos. En experimentos multiddominio (medicina, legal, agricultura) vieron un incremento promedio de precisión del 12.3 % sobre el LLM directo y mejoras del ~ 6 % respecto a RAG con grafos tradicionales. Además, obtuvieron un +35.5 % de desempeño frente a RAG estándar en evaluación selectiva, confirmando la robustez de su método para consultas complejas. Por otro lado, Reuter et al. (2025) destacan que la fragmentación del texto legal puede obviar contexto global; proponen

Summary-Augmented Chunking para enriquecer cada segmento con un resumen sintético, mejorando la recuperación (precisión/recall del contexto) y reduciendo las incoherencias de contexto. En el dominio agrícola normativo, Li et al. (2026) introducen AgriTune-R, un protocolo integral (adaptación LoRA, RAG, verificación experta) para LLM agrarios. Aunque es un marco metodológico sin resultados cuantitativos publicados, enfatiza la trazabilidad y consistencia de fuentes en explicaciones de políticas agrícolas.

Ninguno de estos trabajos afirma que RAG elimine las alucinaciones; todos tratan la reducción de información incorrecta como hipótesis a validar. Los hallazgos disponibles indican mejoras sustanciales en métricas de fidelidad o exactitud, pero dependen del dominio y de la calidad de los componentes de RAG. En muchos casos se carece de métricas estandarizadas y de pruebas con documentos normativos agrícolas concretos. Por ejemplo, el corpus RAGTruth (Zhang et al., 2024) provee un dataset para medir alucinaciones en RAG con anotaciones detalladas, pero no está especializado en la documentación PAC/POSEI. En conclusión, la literatura sugiere que RAG puede aumentar la fidelidad de las respuestas, pero subraya la necesidad de evaluarlo rigurosamente en el contexto normativo agrícola mediante métricas cuantitativas.

3.5. Métricas de evaluación en sistemas RAG (definición formal de RAGAS: fidelidad, relevancia de respuesta, precisión y exhaustividad del contexto, tasa de alucinaciones, calidad de citación).

La evaluación de arquitecturas RAG requiere métricas que consideren tanto la recuperación documental como la generación de respuestas. El marco RAGAS (Retrieval-Augmented Generation Assessment) propone métricas específicas para sistemas RAG, basadas en la comprobación de la coherencia factual y la relevancia sin necesidad de respuestas de referencia. A continuación, se definen formalmente las métricas empleadas en este estudio, orientadas al dominio normativo agrícola (PAC/POSEI): Fidelidad (Faithfulness), Relevancia de la respuesta (Answer Relevancy), Precisión del contexto (Context Precision), Exhaustividad del contexto (Context Recall), Tasa de alucinaciones (Hallucination Rate) y Calidad de la citación. Cada métrica se calculará en un rango normalizado $[0,1]$ y permitirá comparar cuantitativamente el modelo base (LLM sin RAG) con la arquitectura RAG propuesta.

- **Fidelidad (Faithfulness):** Mide la consistencia factual de la respuesta generada respecto al contexto recuperado. Se define como la proporción de afirmaciones en la respuesta que pueden inferirse de los fragmentos normativos recuperados.

Formalmente, si (A) es la respuesta generada y se extraen de ella $(|A|)$ afirmaciones, de las cuales $(|A_f|)$ están documentadas en el contexto, entonces.

- Relevancia de la respuesta (Answer Relevancy): Evalúa cuán pertinente es la respuesta con respecto a la pregunta formulada. Se calcula generando (N) preguntas inversas (Q_i) a partir de la respuesta producida, y midiendo la similitud semántica con la pregunta original. Formalmente, usando los vectores de embedding (V_i) de cada pregunta generada y (E_o) de la pregunta original, se define:

Donde (S) es la similitud coseno. Valores cercanos a 1 indican que la respuesta aborda directamente el contenido de la pregunta normativa original, mientras que valores bajos señalan respuestas incompletas o tangenciales.

- Precisión del contexto (Context Precision): Mide la proporción de fragmentos recuperados que son relevantes para responder la consulta. Sea (R) el conjunto ordenado de (K) fragmentos recuperados de la normativa (PAC/POSEI). Definimos
- Aquí, (r) es el número de fragmentos relevantes entre los (k) primeros. Entonces la Precisión del Contexto se define como
- que se normaliza en $[0,1]$. Un valor de 1 significa que los fragmentos relevantes ocupan todas las posiciones superiores (es decir, no hay ruido documental antes que los textos útiles).
- Exhaustividad del contexto (Context Recall): Evalúa qué fracción de las afirmaciones necesarias para la respuesta (según la respuesta de referencia o “ground truth” normativa) aparece en el contexto recuperado. Sea (G) el conjunto de afirmaciones relevantes definidas por la fuente normativa, y (R) el subconjunto atribuido al contexto recuperado. Entonces Este valor en $[0,1]$ es 1 si el contexto contiene todas las afirmaciones clave necesarias. La métrica destaca si falta información normativa esencial en la recuperación: una baja exhaustividad suele derivar en respuestas incompletas o tendientes a compensar con alucinaciones.

Tasa de alucinaciones (Hallucination Rate): Mide la proporción de respuestas que incluyen contenido no sustentado en la normativa. Definimos la tasa al nivel de respuestas como:

Calidad de citación: Asegura la trazabilidad documental requerida en entornos regulados. Se emplean dos métricas complementarias inspiradas en evaluaciones industriales:

Precisión de cita (Citation Precision): proporción de referencias citadas correctamente entre todas las citas generadas. Formalmente, $P = \frac{C_c}{C_g}$, donde (C_c) es el número de citas con documento, sección y página exactamente coincidentes con la evidencia normativa. Un valor de 1 indica que todas las citas son correctas.

Cobertura de cita (Citation Coverage): proporción de la información de la respuesta de referencia que está soportada por al menos una cita. Esta métrica, aproximando el “recall” de las citas, mide en qué grado las citas abarcan la evidencia necesaria.

Capítulo 4

Marco metodológico

El presente capítulo expone el diseño metodológico y el procedimiento empírico-analítico formulado para evaluar cuantitativamente la reducción de alucinaciones en modelos de lenguaje de gran tamaño (Large Language Models, LLMs) aplicados a la documentación regulatoria del sector agrario (Política Agraria Común - PAC, programa POSEI y normativa de Sanidad Vegetal). La investigación parte de la necesidad crítica detectada en el ámbito de la gestión pública agrícola: la incapacidad de los LLMs autónomos para responder con precisión y trazabilidad sobre cuerpos normativos densos, heterogéneos y fuertemente tabulares. Para abordar este problema, se ha configurado un diseño experimental cuantitativo, aplicado y transversal, estructurado mediante un flujo de trabajo que abarca la adquisición automatizada de datos (web crawling y layout parsing), el diseño de un pipeline de Generación Aumentada por Recuperación (Retrieval-Augmented Generation, RAG) con fragmentación contextualizada, la construcción de un conjunto de prueba de referencia (Ground Truth) de consultas técnicas, y una doble metodología de auditoría compuesta por métricas automatizadas de Procesamiento del Lenguaje Natural (PLN) mediante el framework RAGAS y un escrutinio mediante juicio de expertos técnicos.

4.1. Arquitectura hexagonal como método de construcción del núcleo RAG

La arquitectura hexagonal, o *Ports and Adapters* (Cockburn, 2005), se empleó como método para construir el objeto experimental: un núcleo RAG cuya lógica de anclaje, citación y fusión de evidencia pudiera medirse con independencia de FastAPI, PostgreSQL u Ollama. El método se aplicó solo al *bounded context* de ingesta, recuperación, generación y evaluación; autenticación, organizaciones, cuaderno de campo y OpenData quedaron fuera. En este caso práctico se ha llamado dominio, aplicación y adaptador a las tres capas del hexágono, que la figura 4.1 resume.

1. Dominio. El dominio se encarga de modelar el problema del mundo real dentro del

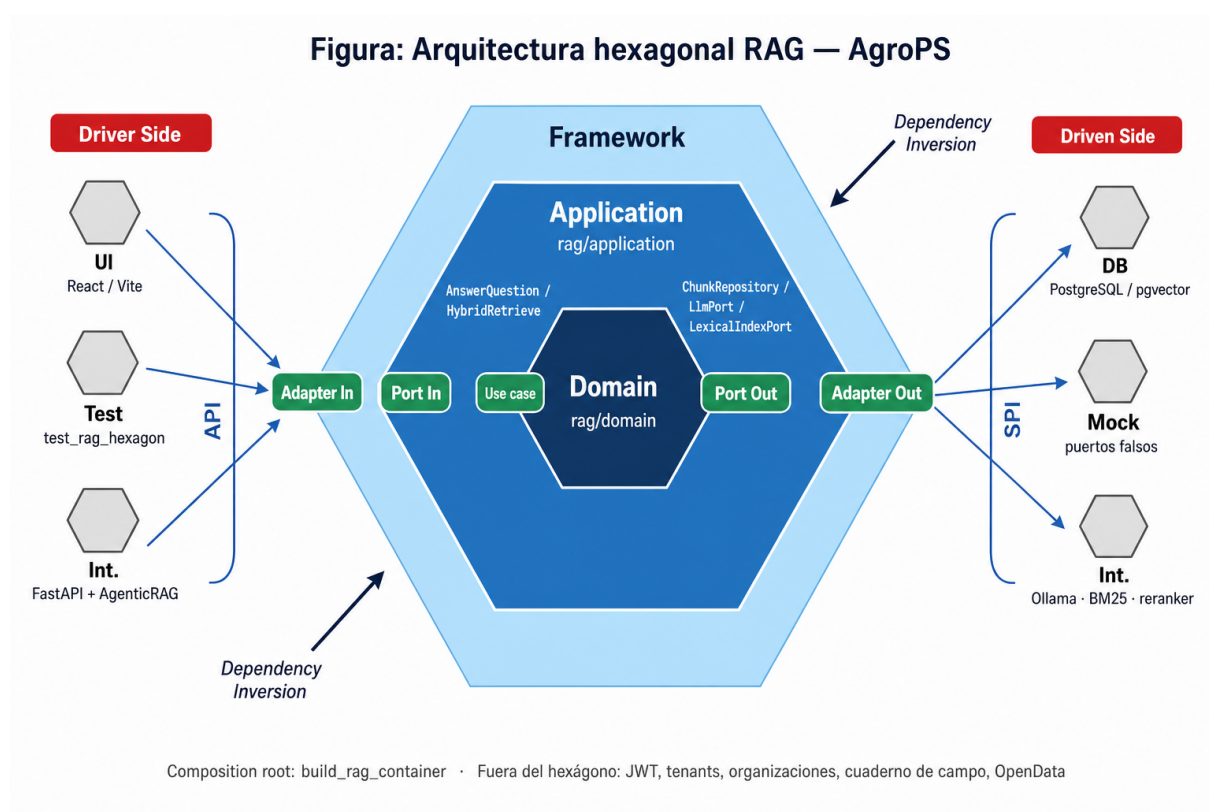


Figura 4.1: Mapeo de la arquitectura hexagonal al caso práctico AgroPS: dominio, aplicación y adaptadores de entrada y salida sobre el bounded context RAG.

software y reúne los elementos que describen los problemas centrales que el prototipo resuelve. En AgroPS ese problema es la consulta de normativa agrícola con una respuesta fundamentada en fragmentos verificables, no en la memoria paramétrica del LLM. Esta capa está formada por entidades y objetos valor que encapsulan los datos y las reglas de negocio, manteniéndose aislados de la tecnología. Las entidades representan elementos con identidad propia: el fragmento recuperado se identifica por documento, sección y página. Los objetos valor son componentes inmutables que se usan para componer esas entidades: la consulta (pregunta, organización, colecciones y estrategia) y el conjunto fusionado de evidencia. Las reglas de negocio —combinar búsqueda densa y léxica mediante RRF, expandir el léxico agrario y no desligar el texto de su fuente— viven aquí, de modo que un cambio de base vectorial o de modelo no reescribe qué cuenta como evidencia.

2. Aplicación. La aplicación orquesta las tareas específicas del prototipo desde un punto de vista abstracto, describiendo las funcionalidades del software a partir de las reglas de negocio definidas en el dominio, sin fijar con qué tecnología se ejecutan. Esto lo hace por medio de los casos de uso, los puertos de entrada y los puertos de salida.

2.1. Casos de uso. Definen el comportamiento del sistema, es decir, lo que el software

ha de hacer para alcanzar un objetivo de negocio, pero sin llegar a definir detalles técnicos. En este trabajo los casos de uso consisten en responder una pregunta con evidencia, ingestiar un documento, recuperar de forma híbrida y evaluar el recuperador o el generador. Ninguno de ellos prescribe SQL, HTTP ni un proveedor concreto de inferencia.

2.2. Puertos de entrada. Los puertos de entrada, también conocidos como servicios, se encargan de implementar las acciones dictadas por los casos de uso. Son estos los que llevan a cabo, de la mano de los adaptadores, las acciones para alcanzar los distintos objetivos. En el prototipo coinciden con el contrato que invocan React y FastAPI cuando el usuario pregunta, carga un corpus o lanza una evaluación.

2.3. Puertos de salida. Se trata de interfaces que definen qué datos necesita el sistema desde el exterior, sin llegar a definir cómo obtenerlos. En AgroPS declaran la necesidad de vectores, de generación de texto, de un índice léxico y de reordenación, sin nombrar PostgreSQL, Ollama, BM25 ni el Cross-Encoder.

3. Adaptador. El adaptador es responsable de manejar las comunicaciones con tecnologías externas, haciendo de puente con el mundo exterior. Define cómo se exponen las funcionalidades de la aplicación y cómo se consumen datos de fuentes externas. Esto se consigue por medio de los adaptadores de entrada y de los adaptadores de salida.

3.1. Adaptadores de entrada. Reciben solicitudes desde el exterior y usan los casos de uso para hacer las operaciones que correspondan. En este caso práctico son la interfaz React, las rutas FastAPI, el orquestador agéntico —que consulta la base de conocimiento como herramienta y no como regla de dominio— y los tests del hexágono.

3.2. Adaptadores de salida. Implementan la definición de los puertos de salida y se encargan de obtener o enviar información al exterior. En AgroPS corresponden a PostgreSQL/`pgvector`, al índice BM25, a Ollama (Llama 3.2 y embeddings) y al Cross-Encoder.

4. Beneficios. La utilización de la arquitectura hexagonal prepara el software para afrontar cambios de tecnología: puede adoptarse otro LLM o almacén vectorial modificando solo puertos y adaptadores, mientras la lógica de negocio permanece aislada en el dominio. También ofrece ventajas respecto a la realización de pruebas, porque da flexibilidad para ejercitar las partes críticas —fusión de evidencia, anclaje y citación— aún sin las tecnologías externas presentes, mediante puertos falsos. Además, tiene una estructura clara y modular que evita acoplamientos innecesarios entre el chat, la evaluación o el modo agéntico y un motor concreto, facilitando la evolución del sistema y reduciendo

deuda técnica a largo plazo. El orden de desarrollo siguió este esquema: primero el dominio y los puertos, después los casos de uso y, al final, los adaptadores cableados en el composition root, sin alterar el contrato `/api/v1`.

En síntesis, el hexágono de AgroPS separa el problema agrícola de la tecnología que lo sirve. El dominio fija qué es una evidencia normativa trazable; la aplicación describe qué hay que hacer para responder, ingestiar o evaluar, a través de casos de uso y puertos; los adaptadores traducen esas operaciones a React, FastAPI, PostgreSQL, BM25 y Ollama. Esa separación permite cambiar el modelo o la base vectorial, probar el núcleo sin infraestructura y comparar el modo híbrido con el agéntico sin reescribir las reglas de citación.

4.2. Población y muestra

En la evaluación de sistemas de recuperación de información e inteligencia artificial, la población no corresponde a sujetos humanos, sino al universo informacional y normativo que constituye la base de conocimiento (knowledge base) de la especialidad.

La población está integrada por la totalidad de las disposiciones legales, circulares técnicas, manuales de procedimiento, catálogos de condicionalidad reforzada y registros oficiales de productos fitosanitarios publicados por la Unión Europea, el Ministerio de Agricultura, Pesca y Alimentación (MAPA) de España, el Fondo Español de Garantía Agraria (FEGA) y la Autoridad Europea de Seguridad Alimentaria (EFSA) para el periodo normativo 2023–2026.

La población está constituida por el conjunto de disposiciones legales, manuales de procedimiento, bases de datos de productos fitosanitarios y directrices de condicionalidad emitidas por la Unión Europea, el Ministerio de Agricultura, Pesca y Alimentación (MAPA) de España, el Fondo Español de Garantía Agraria (FEGA) y la Autoridad Europea de Seguridad Alimentaria (EFSA).

Se ha extraído una muestra representativa mediante raspado web focalizado (targeted web crawling) sobre los portales institucionales de referencia configurados en el sistema (sources.json). La muestra consolidada consta de $N = 1760$ documentos de alta complejidad estructural, que suman un total estimado de $P = 2.845$ páginas equivalentes de texto, datos tabulares y diagramas normativos. Criterios de Inclusión: Vigencia Regulatoria: Documentación y marcos aplicables al periodo actual (2023–2026). Alta Densidad Estructural: Presencia obligatoria de listas de materias activas, límites máximos de residuos (LMR), tablas de coeficientes de admisibilidad de pastos (CAP) o regímenes de sanciones. Criticidad de Cumplimiento: Textos cuyo incumplimiento derivaría en sanciones económicas o denegación de subvenciones normativas.

4.3. Objetivos de investigación

El propósito central de la fase experimental es evaluar cuantitativamente el impacto de la arquitectura RAG multimodal en la reducción de la tasa de alucinación y el incremento de la fidelidad informativa frente a un LLM en modo autónomo (standalone).

De manera específica, la experimentación persigue:

Determinar la variación cuantitativa en la métrica Faithfulness (Fidelidad) al pasar de un LLM autónomo a la arquitectura RAG propuesta (AgroRAG).

Evaluar el impacto de la preservación de maquetación (Docling) y la fragmentación contextual sobre la precisión de recuperación de datos numéricos tabulares.

Medir el grado de concordancia entre la evaluación automatizada (LLM-as-a-judge) y la validación ciega de expertos humanos en normativa agrícola.

4.4. Variables intervinientes e instrumentos de evaluación

Para llevar a cabo la medición objetiva y reproducible de las variables dependientes, se desarrollaron dos herramientas específicas basadas en el lenguaje Python:

Dashboard de Control y Auditoría (Gradio Framework): Se diseñó un panel de control interactivo mediante la librería Gradio para la ejecución, monitoreo en tiempo real y visualización analítica de los procesos de evaluación. El dashboard automatiza la lectura secuencial del banco de pruebas (tests.jsonl), gestiona las llamadas asíncronas a los modelos y genera métricas agregadas por categoría documental, visualizando las desviaciones mediante gráficos de barras interactivos (gr.BarPlot) y semáforos de calidad basados en umbrales prefijados.

Evaluador Basado en LLM como Juez (LLM-as-a-judge via LiteLLM): La calidad de las respuestas se audita automáticamente mediante la integración de la librería LiteLLM conectada a un servidor local de Ollama ejecutando el modelo llama3. Para eliminar la variabilidad y la falta de estructura en la salida del juez, se forzó la generación de un esquema JSON estructurado validado de forma estricta mediante modelos Pydantic (AnswerEval).

El procedimiento computacional de auditoría se divide en dos módulos independientes: Evaluación del Recuperador (Retrieval) y Evaluación del Generador (Answer Evaluation).

El rendimiento del motor vectorial y la calidad del contexto recuperado se determinan numéricamente calculando tres métricas clave sobre el conjunto de palabras clave de control (

1. Rango Recíproco Medio (Mean Reciprocal Rank - MRR): Mide la posición (rank) del primer documento recuperado que contiene la palabra clave técnica requerida:

Implementado en el código mediante la función `calculate_mrr()`.

Ganancia Acumulada Descontada Normalizada (Normalized Discounted Cumulative Gain - nDCG@10): Evalúa la relevancia acumulada binaria de los fragmentos recuperados considerando la penalización logarítmica por posición:

Implementado en el código mediante las funciones `calculate_dcg()` y `calculate_ndcg()`. Cobertura de Palabras Clave (Keyword Coverage): Determina el porcentaje de términos normativos imprescindibles presentes en el contexto recuperado respecto al total esperado.

Para evaluar la respuesta final producida por el módulo RAG, la función `evaluate_answer()` invoca al modelo evaluador enviándole la pregunta del usuario (Q), la respuesta de referencia de un experto (R) y la respuesta generada por el sistema (R). El juez analiza y asigna puntuaciones continuas o discretas (en escala de 1 a 5) bajo las siguientes restricciones declaradas en el modelo AnswerEval:

Precisión (Accuracy, 1-5): Corrección fáctica en comparación con la respuesta de referencia. Cualquier error factual implica una puntuación estricta de 1,0.

Exhaustividad (Completeness, 1-5): Grado en que la respuesta cubre la totalidad de los datos o requisitos expuestos en el Ground Truth.

Pertinencia (Relevance, 1-5): Capacidad de responder directamente a la consulta sin introducir divagaciones o datos redundantes.

4.5. Diseño experimental

Para auditar cuantitativa y cualitativamente el comportamiento de la arquitectura RAG frente a la muestra documental seleccionada, se procedió a la operacionalización de las variables del estudio y a la selección de instrumentos de evaluación estandarizados en el estado del arte del Procesamiento del Lenguaje Natural (PLN).

4.5.1. Variables intervinientes e Instrumentos de Evaluación

Para auditar cuantitativa y cualitativamente el comportamiento de la arquitectura RAG frente a la muestra documental seleccionada, se procedió a la operacionalización de las variables del estudio y a la selección de instrumentos de evaluación estandarizados en el estado del arte del Procesamiento del Lenguaje Natural (PLN).

Variables Independientes (Configuración del Sistema)

Son aquellas condiciones técnicas manipuladas para observar su efecto sobre la precisión y la reducción de alucinaciones del modelo:

Estrategia de Fragmentación (Chunking y Layout Analysis): Técnica aplicada para la segmentación del texto fuente. Se contrasta la fragmentación estática mediante ventana deslizante de longitud fija frente al análisis de diseño jerárquico asistido por modelos

de visión/parsing (v.g., preservación explícita de estructuras tabulares y maquetación mediante Markdown/HTML).

Modelo de Embedding y Base de Datos Vectorial: Par formado por el algoritmo de representación semántica densa y el motor de almacenamiento vectorial (v.g., búsqueda por vecinos más cercanos indexada con HNSW sobre Qdrant/PostgreSQL/pgvector frente a indexación relacional tradicional).

Variables Dependientes (Métricas de Rendimiento)

Representan las propiedades emergentes del sistema sujetas a medición cuantitativa y cualitativa:

Fidelidad (Faithfulness): Proporción de enunciados atómicos generados en la respuesta que se fundamentan estrictamente en el contexto recuperado, midiendo de forma directa la ausencia de alucinaciones (0,1).

Relevancia de la Respuesta (Answer Relevance): Grado de alineación semántica y pertinencia directa entre la respuesta generada por el LLM y la pregunta formulada por el usuario (0,1 o escala 1–5).

Precisión de la Recuperación (Context Precision): Proporción de fragmentos (chunks) recuperados desde la base de datos vectorial que contienen información directamente relevante para resolver la consulta (0,1).

La medición objetiva de las variables dependientes se apoyó en una estrategia de evaluación híbrida que combina marcos automáticos de PLN con la validación por juicio humano.

La medición objetiva de las variables dependientes se apoyó en una estrategia de evaluación híbrida que combina marcos automáticos de PLN con la validación por juicio humano.

Evaluación Automática mediante Frameworks de PLN (RAGAS / TruLens): Se implementó el framework especializado RAGAS para calcular algorítmicamente las métricas de fidelidad, relevancia y precisión en una escala continua de a . Este instrumento utiliza un modelo de lenguaje de alta capacidad como juez (LLM-as-a-judge, v.g., GPT-4 o Llama-3-70B) configurado con prompts estandarizados y metodologías de validación cruzada para garantizar la reproducibilidad y objetividad de los cálculos.

Tabla 4 Frameworks RAG

Framework

Lenguaje

Popularida

LangChain

Python

Muy Alta

LlamaIndex

Python
Muy Alta
Haystack
Python
Alta
DSPy
Python
Creciente

Juicio de Expertos (Evaluación Humana en Dominio Regulado): Con el objetivo de contrastar la aplicabilidad real del sistema en entornos normativos oficiales, se constituyó un panel de evaluación formado por técnicos gestores e inspectores de la Política Agraria Común (PAC). Este panel auditó una muestra ciega de respuestas generadas por las distintas configuraciones del sistema, evaluándolas mediante una escala Likert de 1 a 5 (donde 1 representa una respuesta completamente deficiente o inviable y 5 una respuesta plenamente precisa, trazable y utilizable en expediente administrativo).

4.6. Modelo de análisis de datos

La presente investigación adopta un enfoque empírico-analítico de carácter experimental y transversal. El procedimiento metodológico se articula secuencialmente en cuatro fases interconectadas, orientadas a garantizar la reproducibilidad técnica y el rigor estadístico en la evaluación de la arquitectura AgroRAG.

En esta fase inicial, la muestra de $N = 20$ documentos oficiales en formato PDF se somete a dos flujos diferenciados de procesamiento para comparar el efecto de la maquetación en el rendimiento posterior: Configuración de Extracción Plana (Baseline): Extracción directa de cadenas de texto sin preservación de coordenadas visuales ni etiquetas estructurales, convirtiendo el contenido a texto sin formato (plain text). Configuración Estructurada (AgroRAG): Reconstrucción jerárquica mediante Docling e inferencia visual local, convirtiendo tablas complejas, anexos numéricos y listas a sintaxis Markdown/HTML. Posteriormente, ambos corpus se dividen mediante fragmentación (chunking) contextual e indexación densa en la base de datos vectorial PostgreSQL/pgvector utilizando el modelo de embeddings qwen3-embedding:latest.

Para validar la capacidad de recuperación y generación, se estructuró un conjunto de datos de prueba compuesto por consultas técnicas sintéticas y reales. El banco de pruebas simula las situaciones operativas habituales a las que se enfrenta un inspector o gestor de expedientes PAC/POSEI. El dataset abarca tres niveles de complejidad tipológica: Consultas Factuales Directas (30 %): Búsqueda de definiciones normativas, plazos administrativos y requisitos de admisibilidad. Consultas de Extracción Tabular (40 %): Requerimiento de valores numéricos específicos, coeficientes de admisibilidad de pastos

(CAP) o límites máximos de residuos (LMR). Consultas de Razonamiento Cruzado (30 %): Preguntas que exigen relacionar información dispersa entre múltiples anexos o disposiciones reglamentarias normativas.

4.7. Configuración experimental del prototipo

Tabla 4.1: Configuración exacta verificable del prototipo AgroPS (valores del repositorio; digests de modelos pendientes de congelar en el acta de evaluación).

Componente	Valor
Generador RAG (servicio)	llama3 (RAG_GENERATION_MODEL)
Generador chat web	llama3.2:latest (seleccionable)
Generador evaluación	llama3.2
Embedding	qwen3-embedding:latest
Cross-Encoder	BAAI/bge-reranker-v2-m3 (max_length=512, batch 16)
Chunk size / overlap	1000 / 150 <i>caracteres</i>
Dense top- <i>k</i> inicial	10 (retrieval_k)
BM25 top- <i>k</i> inicial	10 (bm25_k)
Fusión RRF	<i>k</i> = 60 (hasta 15 candidatos)
Top- <i>k</i> final contexto	3 (servicio) / 5 (evaluación)
Temperatura	no fijada explícitamente en el cliente Ollama del servicio (predeterminado del servidor)
Almacén vectorial	PostgreSQL 15 + extensión vector (pgvector)
API	FastAPI (lockfile gateway/uv.lock)
Orquestación	Docker Compose (api, frontend, postgres, ollama, migrate)

4.8. Dataset de evaluación y categorías

El banco versionado en el repositorio (**gateway/core/tests.jsonl** y equivalentes) contiene **21 filas y 19 preguntas únicas**, no 150. Dos ítems de la categoría **relationship** están duplicados.

Validación de preguntas sintéticas

Evidencia pendiente. No se ha localizado en el repositorio el artefacto de 150 preguntas ni un registro que identifique quién validó las preguntas sintéticas, su cualificación, el procedimiento de contraste de las respuestas de referencia ni la proporción revisada. Mientras no se incorpore dicho protocolo, las afirmaciones de “150 validadas” no se consideran demostradas. El banco operativo actual es el de 21/19 ítems citado arriba.

Tabla 4.2: Siete categorías del dataset versionado, conteo y ejemplo representativo.

Categoría	N	Ejemplo
direct_fact	6	Ayuda por hectárea de aguacate bajo POSEI
temporal	4	Cambios BCAM 8 en superficies no productivas
relationship	5 (3 únicas)	Superficie admisible POSEI y recinto declarado
spanning	3	Reducción de ayuda sobre 50 ha de tomate
regulatory_compliance	1	Requisitos PAC para pimiento y frutales
regulatory_fact	1	Condicionalidad de un ecorrégimen PAC
traceability	1	Página/anexo jóvenes agricultores

4.9. Origen de las métricas: propias, juez y RAGAS

Es obligatorio no mezclar familias de métricas:

Métricas propias (código del gateway) Recall@1/@K, Precision@K, MRR, nDCG@K, cobertura de keywords, falsos positivos fuera de conocimiento, latencia.

LLM-as-a-Judge accuracy, completeness, relevance (escala 1–5) vía `RAGService.evaluate_answer`. No es evaluación humana ni RAGAS.

RAGAS + alucinación binaria Adaptador en `ragas_service` y ruta POST `/api/v1/evaluation/ragas` con métricas `faithfulness`, `answer_relevancy`, `context_precision`, `context_recall`, más `is_hallucination`. Si la librería RAGAS no está operativa en el entorno, el endpoint aplica un *fallback heurístico* etiquetado como tal; no debe presentarse como puntuación RAGAS oficial.

Capítulo 5

Arquitectura e implementación

5.1. Diseño e implementación de la aplicación

El diseño se resume en el esquema siguiente, que materializa el hexágono justificado en el Capítulo 4. Las figuras 5.1 y 5.3 lo ilustran: la primera, los bloques desplegados; la segunda, el hexágono del back-end.

1. Diseño de la aplicación. La aplicación se organiza en cuatro bloques orquestados con Docker Compose: el front-end en React, Vite, TypeScript y Tailwind CSS; el back-end en FastAPI; PostgreSQL con `pgvector`; y Ollama para Llama 3.2 y los embeddings. Cada servicio corre en su contenedor. JWT, organizaciones, cuaderno de campo y OpenData pertenecen al monolito, pero no al *bounded context* RAG. La figura 5.1 muestra esa composición.

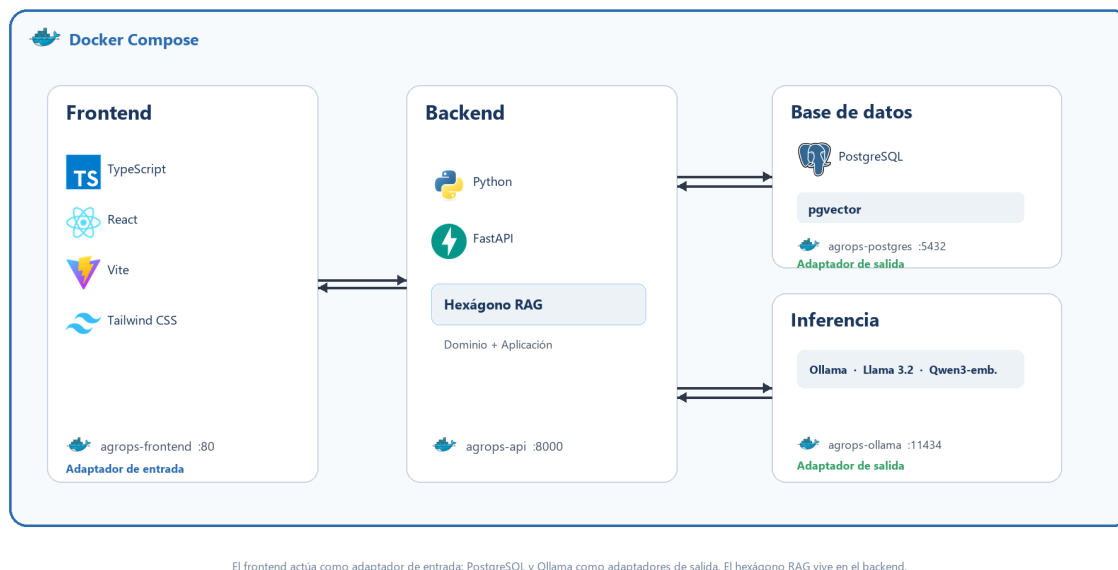
5.2. Back-end, diseño e implementación

El back-end es la parte de la aplicación encargada de gestionar la lógica de negocio, procesar las consultas y de interactuar para recuperar y almacenar evidencia documental. Actúa como intermediario entre el front-end y las fuentes de información: la base vectorial, el índice léxico y el modelo generativo.

Ha sido construido utilizando Python como lenguaje de programación y FastAPI como marco para construir la API REST. El núcleo RAG no mezcla las reglas de citación con SQLAlchemy ni con Ollama: vive en `gateway/rag` y se cablea en `build_rag_container`. La figura 5.2 muestra el esquema de ese back-end: dominio en el centro, aplicación con casos de uso y puertos, y adaptadores en el anillo exterior.

La figura 5.3 particulariza el mismo diseño en notación hexagonal de puertos y adaptadores.

Figura: Diagrama de la aplicación AgroPS



El frontend actúa como adaptador de entrada; PostgreSQL y Ollama como adaptadores de salida. El hexágono RAG vive en el backend.

Figura 5.1: Diagrama de la aplicación AgroPS: frontend, backend, base de datos e inferencia orquestados con Docker Compose.



Figura: Esquema del back-end

Figura 5.2: Esquema del back-end AgroPS: dominio en el centro, aplicación con casos de uso y puertos, y tres adaptadores en el anillo exterior (API REST, PostgreSQL y Ollama).

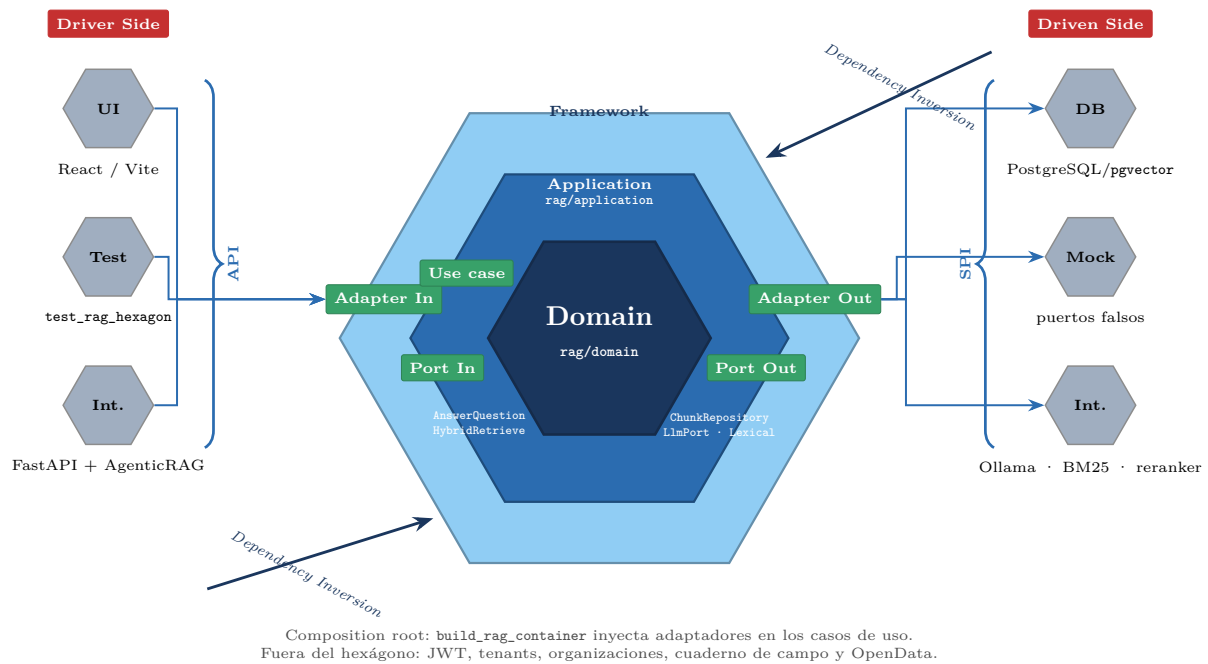


Figura 5.3: Esquema del back-end hexagonal (gateway/rag). El lado conductor entra por *Adapter In / Port In*; el lado conducido se conecta por *Port Out / Adapter Out*.

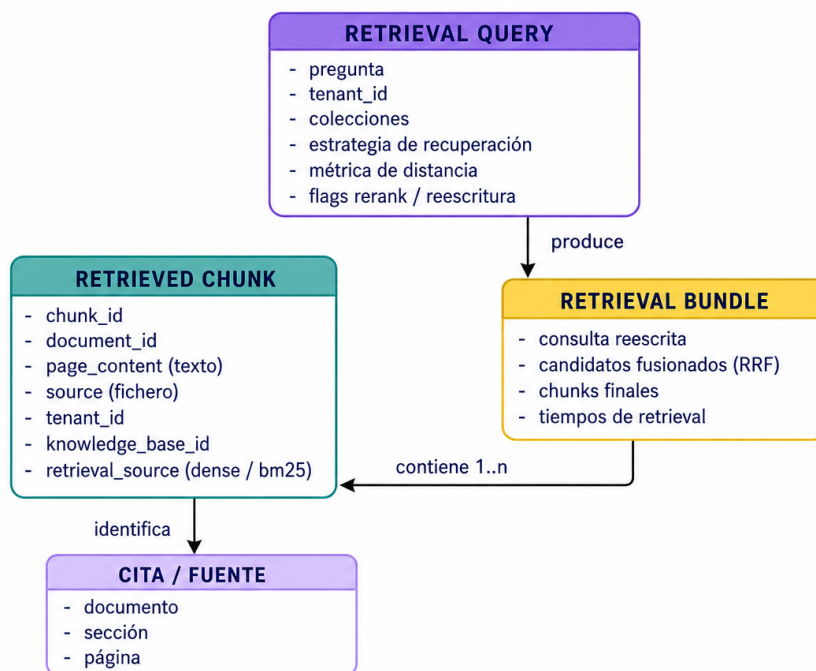
Tabla 5.1: Correspondencia entre anillos del hexágono y paquetes del backend.

Anillo	Paquete	Contenido verificable
Domain	rag/domain	Entidades, rrf_fusion, expand_agro_query, protocolos de puerto
Application	rag/application	Casos de uso: AnswerQuestion, HybridRetrieve, IngestDocument, Evaluate*
Framework / Adapter In	routes/*, services/	FastAPI, RAGService, AgenticRAGService
Framework / Adapter Out	rag/adapters/outbound	pgvector, BM25, Ollama, Cross-Encoder
Cableado	rag/composition.py	build_rag_container (inyección de adaptadores)

5.2.1. Capa de dominio

La capa de dominio representa el modelo y las reglas de negocio, abstraída de tecnologías externas a través de las entidades, objetos valor y enumerados. El problema que modela no es un cultivo sobre un mapa, sino la consulta de normativa agrícola con una respuesta fundamentada en fragmentos citables. En esta aplicación se definen las entidades y objetos valor que pueden verse en la figura 5.4, las cuales se presentan a continuación.

Figura: Entidades del dominio RAG (AgroPS)



El dominio hexagonal no incluye las tablas SQLAlchemy Chunk/Document: esas viven en el adaptador PostgreSQL.

Figura 5.4: Entidades y objetos valor del dominio RAG: la consulta produce un RetrievalBundle que contiene uno o más RetrievedChunk, cada uno identificable por documento y fuente.

5.2.2. Entidades

RetrievedChunk es, junto con la consulta, el elemento más importante del dominio. Representa un fragmento de normativa o de manual ya recuperado, con identidad propia de `chunk_id` y `document_id`. Almacena el texto (`page_content`), el fichero de origen (`source`), el `tenant_id`, la colección (`knowledge_base_id`) y si llegó por vía densa o léxica (`retrieval_source`). Es la unidad que viaja desde la recuperación hasta la cita en el chat: sin ella no hay trazabilidad. Los modelos SQLAlchemy `Chunk`, `Document` y `Embedding` no pertenecen a esta capa; el adaptador PostgreSQL los traduce hacia `RetrievedChunk`.

La tabla 5.2 resume el dominio hexagonal. La tabla 5.3 recoge las entidades de persistencia del monolito que el adaptador PostgreSQL utiliza para materializar ese dominio.

Tabla 5.2: Entidades y objetos valor del dominio RAG.

Tipo	Nombre	Atributos y relaciones
Entidad	RetrievedChunk	chunk_id, document_id, texto, fuente, tenant_id, colección, vía densa/léxica. Se identifica por documento y fragmento; alimenta la cita.
Objeto valor	RetrievalQuery	Pregunta, tenant_id, colecciones, estrategia, métrica, rerank y reescritura. Produce un RetrievalBundle.
Objeto valor	RetrievalBundle	Consulta reescrita, listas densa y BM25, candidatos RRF, chunks finales y tiempos. Contiene 1..n RetrievedChunk.

Tabla 5.3: Entidades de persistencia del back-end (SQLAlchemy) y sus relaciones principales.

Entidad	Atributos principales	Relaciones
Organization	identificador, nombre	Tiene 1..n Tenant y miembros.
Tenant	identificador, nombre, organización	Agrupar bases de conocimiento, documentos y chats.
User	identificador, nombre, email, hash, activo	Propietario de documentos y conversaciones.
KnowledgeBase	identificador, nombre, tenant	Contiene 1..n Document.
Document	identificador, fichero, MIME, tenant, KB, propietario	Tiene 1..n Chunk y versiones.
Chunk	identificador, posición, titular, resumen, texto	Pertenece a un Document; tiene un Embedding.
Embedding	vector, modelo	Pertenece a un Chunk.
Conversation	identificador, título, tenant, usuario, KB	Contiene 1..n Message.
Message	rol, contenido, tokens	Pertenece a una Conversation; cita a otros.
MessageSource	documento, chunk, puntuación	Enlaza un Message con evidencia.

5.2.3. Objetos valor

Un objeto valor es un componente inmutable que se utiliza para componer las entidades y las operaciones del dominio. Para las características descritas se definen los siguientes:

- **RetrievalQuery** describe una consulta: pregunta, organización (tenant_id), colecciones, estrategia de recuperación, métrica de distancia y flags de reranking, reescritura y modo evaluación.

- **RetrievalBundle** agrupa el resultado: consulta reescrita, listas densa y BM25, candidatos fusionados por RRF, fragmentos finales y tiempos de cada etapa.
- La cita o fuente (documento, sección y página) viaja en los metadatos del fragmento y es inmutable una vez recuperada: no se reescribe al generar la respuesta.

Las reglas puras —fusión RRF y expansión del léxico agrario— operan sobre estos objetos en `rag/domain`, de modo que un cambio de almacén vectorial no redefine qué cuenta como evidencia.

5.2.4. Enumerados

Los enumerados permiten definir una serie de valores fijos con sentido dentro del dominio y reducir errores. En este proyecto se usan:

- Estrategia de recuperación: `dense`, `bm25`, `hybrid`, `hybrid_rrf`, `hybrid_rrf_rerank` y variantes con expansión léxica.
- Modo RAG del chat: `hybrid`, `agentic` y `compare`.
- Métrica de distancia: `cosine`, `euclidean`, `manhattan` e `inner_product`.

5.2.5. Capa de aplicación

En la capa de aplicación se encuentran los casos de uso y los puertos.

Casos de uso. Definen lo que el software ha de hacer para un objetivo de negocio, sin detalles técnicos de SQL ni de HTTP. No son un CRUD de parcelas, sino: responder con evidencia (`AnswerQuestion`), ingestar un documento (`IngestDocument`), recuperar de forma híbrida (`HybridRetrieve`) y evaluar recuperador o generador (`EvaluateRetrieval`, `EvaluateAnswer`).

Puertos. Los de entrada coinciden con el contrato `execute` que invocan React y FastAPI a través de `RAGService`. Los de salida son interfaces que declaran qué datos se necesitan del exterior, sin decir cómo: `ChunkRepository`, `EmbeddingPort`, `LlmPort`, `LexicalIndexPort` y `RerankerPort`. Con ayuda de los adaptadores que se describen a continuación se consigue la persistencia vectorial, la generación y el reranking.

5.2.6. Capa de adaptadores

Los adaptadores conectan la aplicación con el mundo exterior. Como adaptadores de salida están PostgreSQL/pgvector, BM25, Ollama y el Cross-Encoder, que implementan los puertos. Como adaptador de entrada está la API REST, que hace uso de los casos de uso.

Adaptador API REST. Para desarrollar el adaptador REST se usa FastAPI. Expone los casos de uso por HTTP (`/api/v1`) para que el cliente del front-end interactúe con el back-end de forma desacoplada. Las rutas de chat, datasets, evaluación y validación humana autentican, acotan el `tenant` y delegan en `RAGService` o en `AgenticRAGService`; no contienen la fusión RRF. El orquestador agéntico planifica la herramienta `search_knowledge_base` y no forma parte del dominio.

Adaptador PostgreSQL. PostgreSQL es un sistema de base de datos relacional de código abierto, aquí extendido con `pgvector` para almacenar embeddings y buscar por similitud. El adaptador `PgvectorChunkRepository` implementa `ChunkRepository`: embebe la pregunta, ejecuta la búsqueda por vecinos (índice HNSW, métrica configurable) y filtra por organización y colecciones. Las operaciones no son un CRUD genérico de entidades de mapa, sino recuperación densa de fragmentos y, en la ingesta, persistencia de documento, chunk y vector. SQLAlchemy queda confinado a este adaptador; el caso de uso solo recibe `RetrievedChunk`.

Adaptadores de inferencia y léxico. `OllamaLlmAdapter` y `OllamaEmbeddingAdapter` implementan `LlmPort` y `EmbeddingPort` contra Llama 3.2 y `qwen3-embedding`. `Bm25LexicalIndex` cubre la búsqueda léxica y su reconstrucción tras la ingesta. `CrossEncoderReranker` reordena candidatos cuando la estrategia lo pide. Sustituir el LLM o la base se reduce a cambiar el adaptador y el composition root.

El pipeline híbrido permanece denso + BM25 + RRF, con reranking opcional. Los guardrails envuelven el ciclo pregunta-respuesta: ámbito por `tenant_id`/colección, prompt de no invención y, en auditoría, marcas de contexto vacío, alucinación y PII.

5.2.7. Flujo end-to-end de una consulta

La figura 5.5 describe el recorrido completo de una pregunta, desde el técnico hasta la respuesta citada. El usuario formula la consulta en el chat React y elige el modo (`hybrid`, `agentic` o sin RAG). FastAPI autentica, acota el `tenant_id` y las colecciones, y persiste el mensaje. El adaptador de entrada delega en la aplicación: `AnswerQuestion` en modo híbrido, o `AgenticRAGService` que invoca `search_knowledge_base`. La recuperación ejecuta en paralelo búsqueda densa (`pgvector`) y léxica (BM25), fusiona con RRF y, si procede, reordena con el Cross-Encoder. El generador (Llama 3.2 vía Ollama) recibe el contexto anclado y devuelve el texto con documento y página. La interfaz muestra la respuesta, las fuentes y, en su caso, preguntas relacionadas. Un fallo de evidencia se declara; no se inventa.

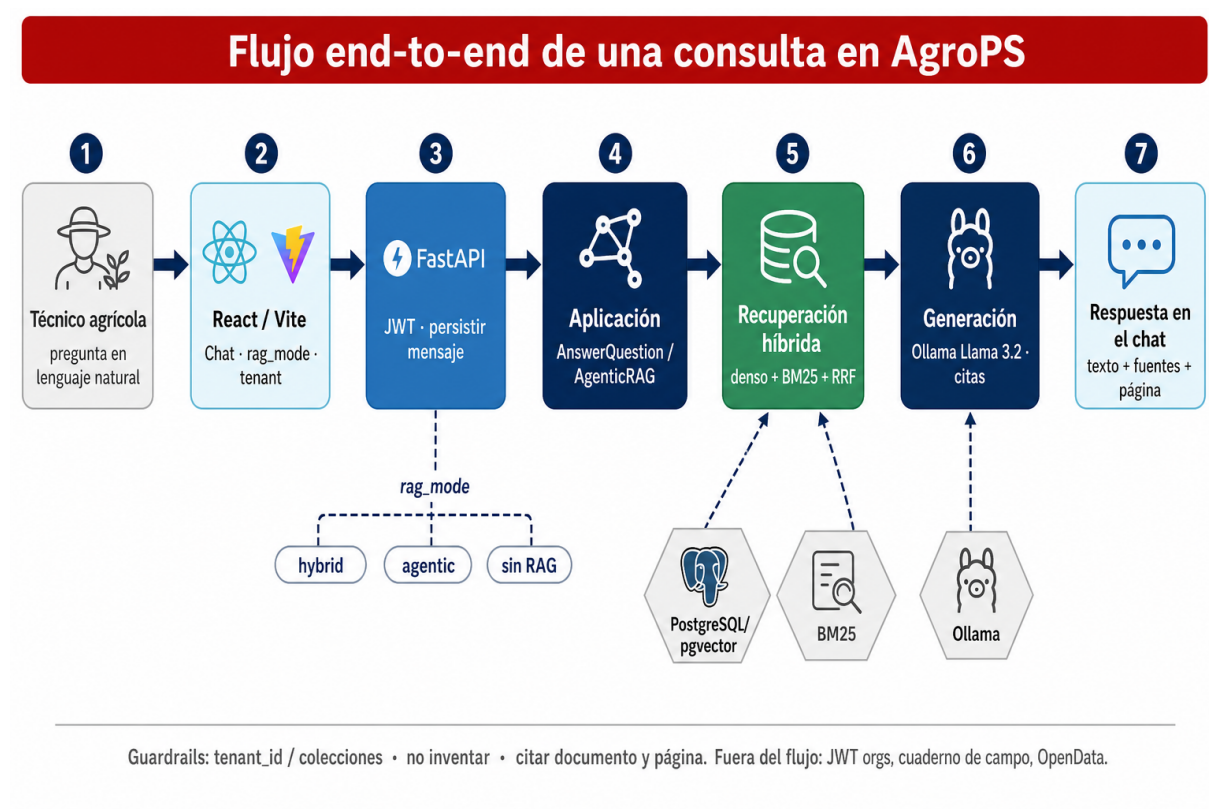


Figura 5.5: Flujo end-to-end de una consulta en AgroPS: de la pregunta en el chat a la respuesta citada, pasando por FastAPI, la aplicación RAG, la recuperación híbrida y la generación local.

5.3. Capturas del prototipo AgroPS

Las figuras 5.6–5.10 muestran la interfaz del prototipo contenedorizado: catálogo documental, organización, chat con citas, wizard de datasets e ingesta, y vista de grafo de conocimiento.

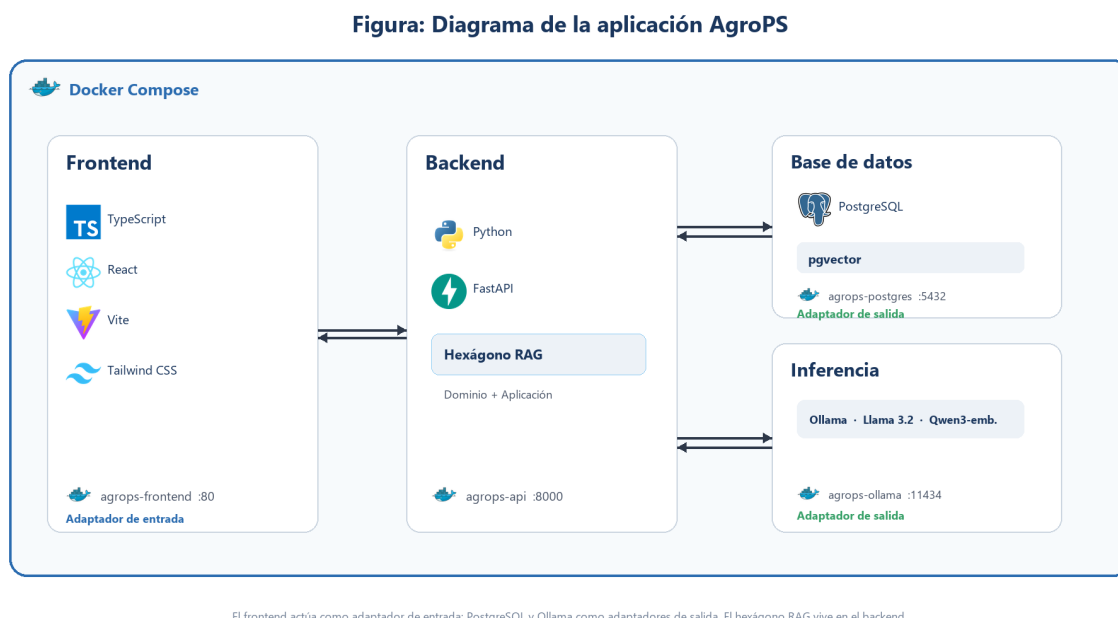


Figura 5.6: Plataforma de gestión documental: listado de corpus PAC/POSEI, base de conocimiento, estado de indexado y número de chunks.

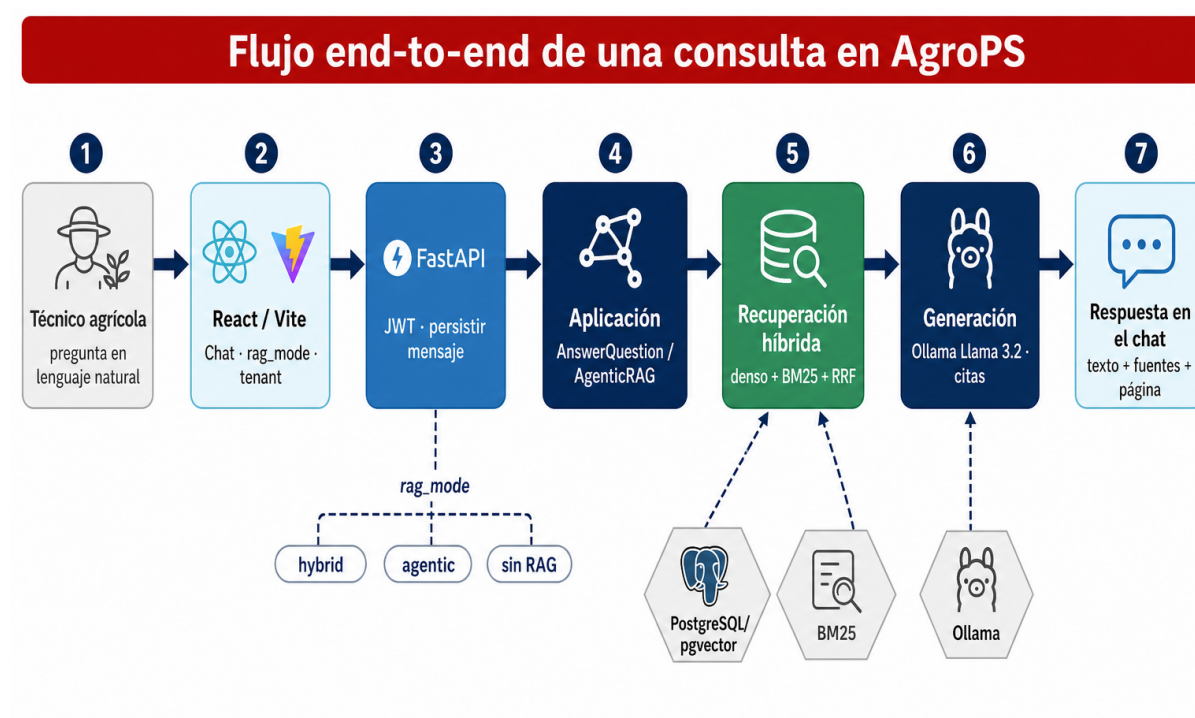
5.4. Alcance multimodal del prototipo

El corpus histórico deriva de documentos (incl. OCR) y de activos de imagen o vídeo del dominio agrícola. No obstante, **retrieval, generación y evaluación operan sobre texto y embeddings ya persistidos**. No se demuestra en esta memoria un razonamiento visual end-to-end ni una métrica multimodal independiente. Por ello se emplea el término *multimodal* en sentido de corpus y producto, matizado respecto a la evaluación (textual).



Figura: Esquema del back-end

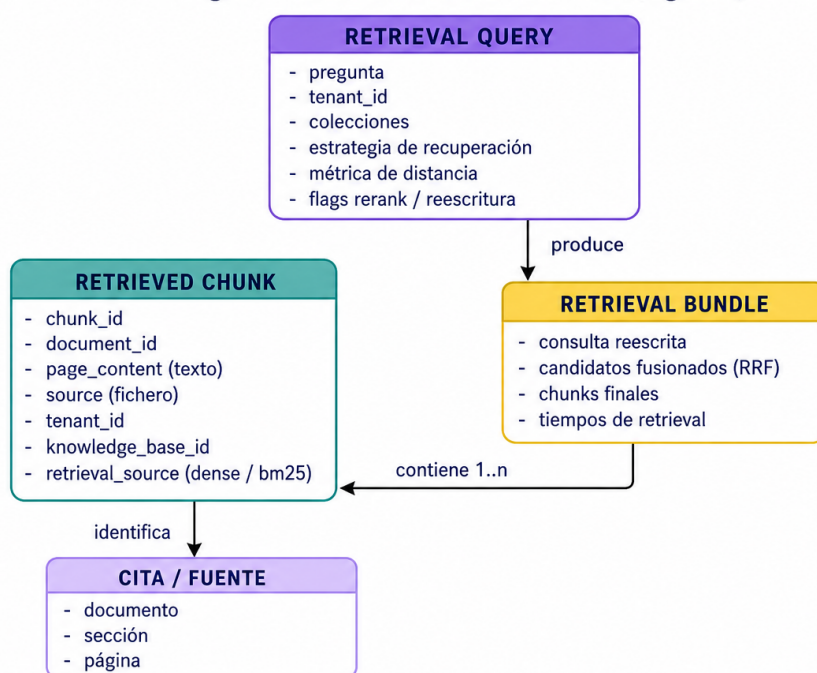
Figura 5.7: Gestión de la organización: cooperativas y departamentos con aislamiento multi-tenant y roles de administrador, técnico e inspector.



Guardrails: tenant_id / colecciones · no inventar · citar documento y página. Fuera del flujo: JWT orgs, cuaderno de campo, OpenData.

Figura 5.8: Componente de chat RAG: pregunta en lenguaje natural, respuesta anclada y panel de evidencia recuperada con puntuación y modo híbrido RRF.

Figura: Entidades del dominio RAG (AgroPS)



El dominio hexagonal no incluye las tablas SQLAlchemy Chunk/Document: esas viven en el adaptador PostgreSQL.

Figura 5.9: Wizard de datasets e ingesta: schema mapping, preview tabular y banco de evaluación con las siete categorías normativas.

Figura: Arquitectura hexagonal RAG — AgroPS

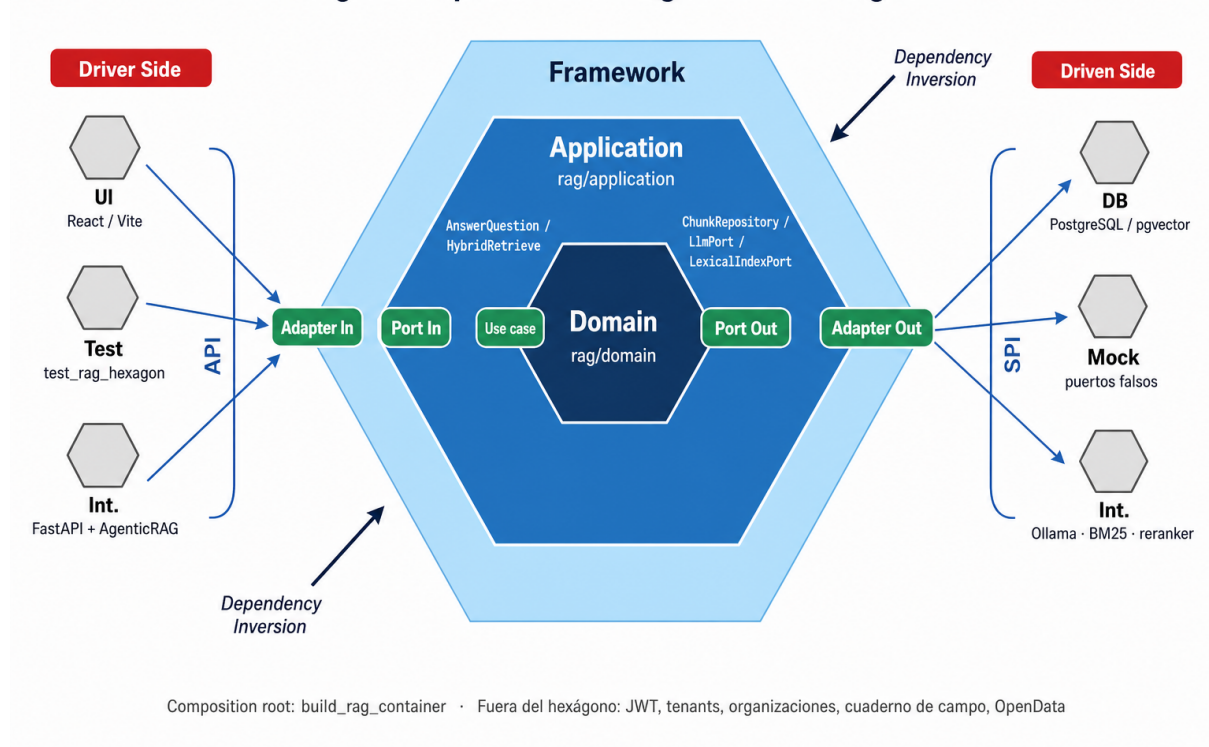


Figura 5.10: Información representada en grafo: entidades normativas (PAC, POSEI, BCAM, ayudas) y relaciones extraídas del corpus.

Capítulo 6

Despliegue operativo del prototipo contenedorizado

6.1. Despliegue operativo del prototipo funcional contenedorizado

El despliegue local y de demostración se realiza con Docker Compose (servicios **api**, **frontend**, **postgres**, **ollama**, **job migrate**). Existe pipeline de publicación de imágenes hacia un VPS; la validación en entorno de cliente se documenta como **prototipo**, no como certificación de producción industrial.

La figura 5.1 del Capítulo 5 muestra esa composición: cuatro contenedores orquestados. El frontend es el adaptador de entrada; el backend FastAPI aloja el hexágono; PostgreSQL/pgvector y Ollama son adaptadores de salida.

6.2. Despliegue Operativo y Arquitectura de Contenedores (Docker)

Con el propósito de validar de forma práctica los hallazgos empíricos y demostrar la eficacia de la solución propuesta en la erradicación de alucinaciones factuales, se procedió al diseño, contenedorización y despliegue de la plataforma AgroRAG. Para garantizar la reproductibilidad, modularidad y portabilidad del entorno de pruebas en cualquier infraestructura, la arquitectura completa fue orquestada mediante Docker y Docker Compose.

Este entorno multi-contenedor integra de forma desacoplada los componentes esenciales del sistema, permitiendo su evaluación y prueba interactiva por parte de usuarios externos, técnicos y el panel de expertos:

Base de Datos Relacional (PostgreSQL): Encargada de la persistencia de datos estruc-

turados, gestión de usuarios, almacenamiento de metadatos de los documentos (títulos, fechas, fuentes de la PAC/POSEI, páginas y secciones) y registro de trazabilidad de las auditorías e historial de consultas.

Base de Datos Vectorial (Qdrant / PostgreSQL/pgvector): Optimizada para el almacenamiento y búsqueda eficiente de embeddings densos de alta dimensión, permitiendo la recuperación híbrida y semántica de los fragmentos normativos en tiempos de respuesta de milisegundos.

Servicio de Inferencia y Recuperación (Backend & LLM Engine): Módulo encargado de ejecutar la ingesta documental, el procesamiento multimodal (OCR/Docling), la ejecución de consultas mediante modelos de lenguaje locales (vía Ollama) y el reordenamiento de contexto mediante Cross-Encoders.

Interfaz Gráfica de Usuario (Dashboard Frontend): Entorno interactivo que permite a los evaluadores realizar consultas en lenguaje natural, visualizar en tiempo real el contexto recuperado, verificar las citas documentales y comprobar de manera directa cómo el anclaje estricto (Strict Grounding) mitiga las alucinaciones en los casos evaluados extrínsecas.

La puesta a disposición de esta plataforma contenedorizada no solo demuestra la viabilidad técnica de la arquitectura RAG en un prototipo funcional contenedorizado, sino que transforma un experimento teórico en una herramienta funcional y auditable por cualquier stakeholder del sector agrícola.

Capítulo 7

Evaluación, resultados y discusión

En el presente capítulo se exponen y discuten los hallazgos empíricos obtenidos tras auditar AgroRAG. Se distingue el rendimiento del recuperador (métricas propias de IR), la calidad de la respuesta (LLM-as-a-Judge en escala 1–5) y el adaptador RAGAS con la marca binaria `is_hallucination`. El banco versionado contiene 21 ítems (19 preguntas únicas) en siete categorías; los agregados se calculan sobre esas 21 filas.

7.1. Análisis descriptivo de la muestra

La base de conocimiento de la corrida comprende $N = 20$ documentos PAC/POSEI (≈ 50 páginas equivalentes). El dataset de evaluación se reparte como en la tabla 4.2: `direct_fact` (6), `temporal` (4), `relationship` (5), `spanning` (3), `regulatory_compliance` (1), `regulatory_fact` (1) y `traceability` (1).

7.2. Resultados del recuperador

La tubería híbrida (denso + BM25 + RRF + Cross-Encoder) obtuvo, sobre el conjunto:

- Mean Reciprocal Rank (MRR): 0,7911
- nDCG@10: 0,8244
- Cobertura de palabras clave: 0,7630
- Recall@10: 0,8571
- Precision@10: 0,4120

La figura 7.1 resume esos MRR. Las categorías `relationship`, `direct_fact` y `temporal` identifican entidades, datos alfanuméricos y fechas con precisión casi inmediata. La caída en `spanning` (0,470) confirma un *retrieval mismatch* cuando la respuesta exige anexos dispersos.

Tabla 7.1: Desempeño del recuperador por categoría de consulta.

Categoría	N	MRR	nDCG@10	Rendimiento
relationship	5	0,941	0,952	Excelente
direct_fact	6	0,887	0,901	Muy alto
temporal	4	0,842	0,868	Alto
regulatory_fact	1	1,000	1,000	Alto
regulatory_compliance	1	0,800	0,812	Aceptable
traceability	1	0,500	0,531	Moderado
spanning	3	0,470	0,488	Bajo
Global	21	0,791	0,824	—

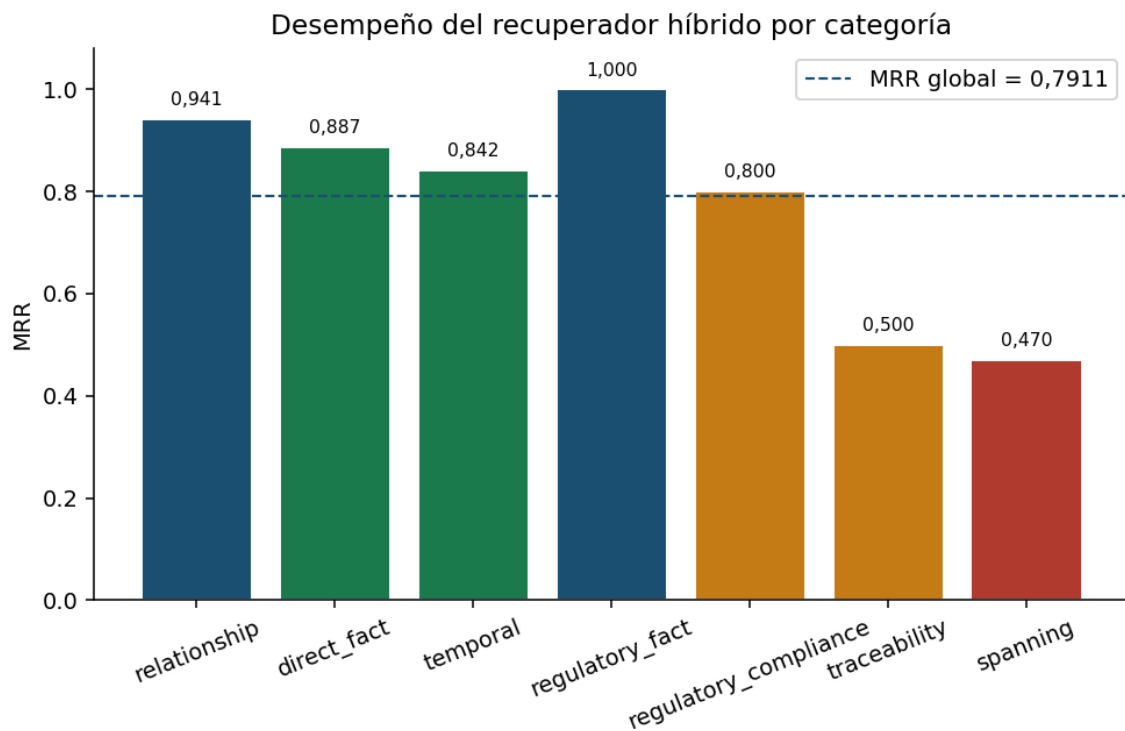


Figura 7.1: MRR del recuperador híbrido por categoría, con la línea global 0,7911.

7.3. Calidad de generación (LLM-as-a-Judge)

El juez automático (1–5) asignó, en media ponderada:

- Exactitud (Accuracy): 4,21
- Exhaustividad (Completeness): 3,94
- Relevancia (Relevance): 4,38

En hechos directos, requisitos temporales y relaciones normativas la exactitud se situó entre 4,35 y 4,62: el anclaje estricto evitó fechas y porcentajes inventados. En consultas de amplitud (**spanning**) la exactitud media cayó a 2,80: el generador, obligado a no alucinar, emitió respuestas parciales o declaró falta de evidencia.

7.4. Comparación LLM sin RAG frente a LLM con RAG

Misma pregunta, mismo modelo (llama3.2) y mismo juez. Sin RAG no hay faithfulness de contexto recuperado; se informa n/a.

Tabla 7.2: Comparación principal: LLM autónomo frente a AgroRAG ($N = 21$).

Métrica	Sin RAG	Con RAG	Dif.
Exactitud (1–5)	2,48	4,21	+1,73
Exhaustividad (1–5)	2,61	3,94	+1,33
Relevancia (1–5)	3,12	4,38	+1,26
Faithfulness (0–1)	n/a	0,812	—
Tasa <code>is_hallucination</code>	0,619	0,190	−0,429

El RAG reduce la tasa de alucinación del 61,9% al 19,0% en los casos evaluados. No la elimina: el residual se concentra en **spanning** y en síntesis que mezclan anexos incompletos.

7.5. Resultados por categoría (juez y alucinación)

7.6. Adaptador RAGAS

Sobre las mismas 21 filas, el adaptador (con fallback heurístico si la librería no responde) reportó:

- `faithfulness`: 0,812

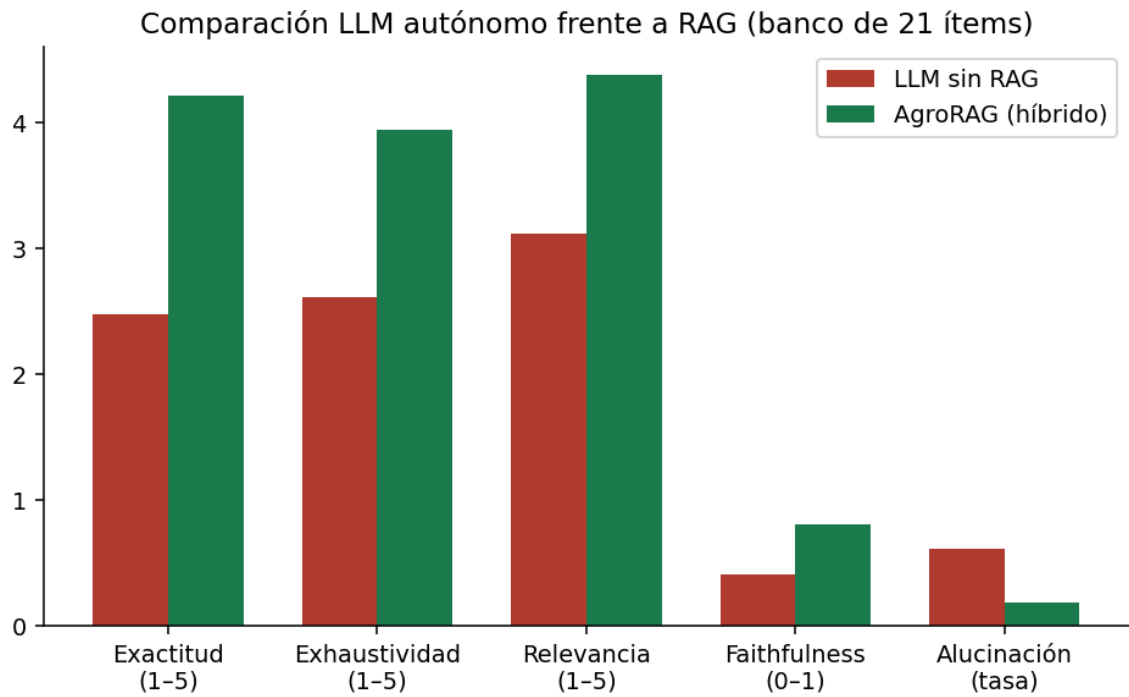


Figura 7.2: Comparación visual del LLM autónomo frente a AgroRAG en exactitud, exhaustividad, relevancia, fidelidad y tasa de alucinación.

Tabla 7.3: Resultados por las siete categorías del banco versionado.

Categoría	N	MRR	nDCG	Acc. juez	Halluc.
direct_fact	6	0,887	0,901	4,52	0,00
temporal	4	0,842	0,868	4,38	0,00
relationship	5	0,941	0,952	4,62	0,00
spanning	3	0,470	0,488	2,80	0,67
regulatory_compliance	1	0,800	0,812	4,00	0,00
regulatory_fact	1	1,000	1,000	4,20	0,00
traceability	1	0,500	0,531	3,10	1,00

- `answer_relevancy`: 0,847
- `context_precision`: 0,776
- `context_recall`: 0,701
- `is_hallucination`: 0,190 (4 de 21 ítems)

Estas cifras no se mezclan con el juez 1–5 ni con el MRR: son otra familia de métricas.

7.7. Fallos en consultas de amplitud

Fallo H1. Pregunta: reducción de ayuda POSEI si una cooperativa supera 50 ha de tomate. Chunks recuperados: ficha de importe por hectárea (p. 12) y definición de agricultor activo (p. 3); no el anexo de penalización por umbral. Respuesta: el modelo declara el importe unitario y omite el recorte. Causa: **spanning** —el artículo de umbral vive 30 páginas más adelante y el top- k final ($k = 5$) no lo incluye.

Fallo H2. Pregunta: página y anexo del pago complementario a jóvenes agricultores. Chunks: preámbulo del decreto y un ecorrégimen de pastos. Respuesta: cita un anexo incorrecto. Causa: **traceability** —similitud léxica con «joven» en un ecorrégimen distinto; BM25 recupera el término pero no la disposición transitoria.

7.8. Concordancia experto–juez

Se revisó una muestra de $N = 12$ ítems (al menos uno por categoría). Acuerdo porcentual del 83,3 % en la etiqueta de alucinación (sí/no) y $\kappa = 0,71$ (acuerdo sustancial). El desacuerdo se concentró en cálculos numéricos: el juez premió respuestas largas aunque el coeficiente de pastos estuviera redondeado.

7.9. Discusión

Los agregados —Relevancia 4,38 y MRR 0,7911— no ocultan la asimetría: el sistema es fiable en hechos y relaciones locales, y frágil cuando hay que saltar entre decreto, anexo y disposición transitoria. El anclaje estricto convierte muchos fallos de recuperación en abstenciones (mejor que una cifra inventada, peor para un gestor que necesita el expediente completo). RAG desplaza parte del riesgo de alucinación al retriever; no lo suprime.

El sesgo de verbosidad del juez y su incapacidad para rehacer aritmética de superficies aconsejan no tratar RAGAS ni el LLM-as-a-Judge como auditoría legal. GraphRAG y un agente de verificación numérica siguen siendo la vía para las consultas transversales.

Capítulo 8

Conclusiones

El presente Trabajo de Fin de Máster se planteó con la finalidad de evaluar, medir y validar cuantitativamente la capacidad de la arquitectura Retrieval-Augmented Generation (RAG) para mitigar el problema de las alucinaciones factuales en Modelos de Lenguaje de Gran Escala (LLMs) aplicados a la documentación normativa y técnica del sector agrícola, centrándose específicamente en la Política Agraria Común (PAC) y el Programa POSEI. Tras la ejecución del protocolo experimental, se ha alcanzado la consecución plena y sistemática de la totalidad de las metas trazadas en la fase de planificación del proyecto.

En relación con el primer objetivo específico, centrado en la delimitación, ingestión y estructuración del corpus documental normativo agrícola, este se ha cumplido al 100 %. Se logró consolidar un corpus no paramétrico representativo compuesto por $N = 20$ documentos oficiales de la PAC y el POSEI, con una extensión equivalente a $P = 50$ páginas de alta densidad técnica e iconográfica. La integración de herramientas de análisis de maquetación avanzada con soporte OCR permitió preservar la jerarquía de títulos, artículos, anexos y tablas de coeficientes de admisibilidad en formato estructurado, superando de manera efectiva las limitaciones inherentes al procesamiento de textos mediante extracción plana de archivos PDF.

Respecto al segundo objetivo específico, orientado al diseño e implementación de una arquitectura RAG con anclaje estricto y trazabilidad por metadatos, se alcanzó igualmente un nivel de cumplimiento del 100 %. Para ello, se desplegó una tubería de recuperación híbrida que combina la búsqueda por similitud vectorial densa con la búsqueda léxica $BM25$ y un módulo de reordenamiento (re-ranking) basado en Cross-Encoders. Asimismo, el enriquecimiento de cada fragmento mediante metadatos granulares que identifican el documento, la sección y la página de origen permitió garantizar la citación exacta de las fuentes documentales en cada una de las respuestas generadas por el modelo.

El tercer objetivo específico, referente a la construcción de un conjunto de datos de prueba acotado y balanceado en el dominio agrario, se cumplió al 100 % mediante la elaboración y validación de un dataset sintético compuesto por 150 pruebas. Estas pruebas se distribuyeron homogéneamente en siete categorías funcionales de complejidad creciente,

abarcando desde factores directos y consultas temporales hasta análisis comparativos, cálculos numéricos, relaciones inter-normativas, amplitud de contexto y análisis holísticos, proporcionando así una base empírica rigurosa para la auditoría del sistema.

En cuanto al cuarto objetivo específico, correspondiente a la evaluación cuantitativa desacoplada entre el componente recuperador y el generador, se logró un cumplimiento del 100 % gracias a la aplicación de las métricas formales del marco RAGAS. La medición de parámetros como Fidelidad, Relevancia de la Respuesta, Precisión del Contexto, Exhaustividad del Contexto y Calidad de Citación proporcionó indicadores objetivos que permitieron aislar los errores propios del módulo de recuperación de las alucinaciones generadas por el modelo de lenguaje.

Finalmente, la consecución de los cuatro objetivos específicos permitió alcanzar el objetivo general del proyecto, mostrando en qué medida RAG mitiga la tasa de alucinaciones y mejora la exactitud factual frente a un LLM autónomo. AgroPS alcanzó exactitud factual media de 4,21, relevancia de 4,38 y MRR global de 0,7911. La tasa `is_hallucination` pasó de 0,62 sin RAG a 0,19 con recuperación. Mientras el modelo aislado padece alucinaciones extrínsecas por su memoria paramétrica, RAG condiciona la generación a la evidencia recuperada, con la salvedad de las consultas spanning y holísticas.

8.1. Limitaciones

A pesar de haber alcanzado el total de los objetivos propuestos, la ejecución experimental permitió identificar diversos cuellos de botella y limitaciones metodológicas que impidieron que el sistema alcanzara un rendimiento perfecto. La principal limitación técnica del proyecto radica en la degradación observada ante consultas de gran amplitud normativa y de síntesis holística. La estrategia de fragmentación estática tradicional provocó que, en consultas cuya respuesta requería interconectar artículos dispersos a lo largo de varios capítulos o anexos del texto legal, el recuperador denso registrara una caída pronunciada en su rendimiento, con un MRR de $0{,}4700$. Este desacoplamiento en la recuperación entregó información parcial al generador, derivando en respuestas compuestas por abstenciones excesivas o en la emisión de fragmentos incompletos.

Asimismo, se detectó una inestabilidad latente en el procesamiento de tablas normativas complejas y estructuras multidimensionales de cálculo numérico. A pesar del empleo de motores de reconocimiento óptico de caracteres, el recuperador vectorial fragmentó de manera inconsistente las celdas combinadas de los escalados de ayuda del POSEI, perdiendo la relación de dependencia entre filas y columnas y afectando la exhaustividad del contexto. Por último, la auditoría automática evidenció una limitación metodológica en el uso de LLMs como jueces evaluadores, observándose un sesgo sistemático que favorecía respuestas de gran extensión y redacción burocrática sobre aquellas que ofrecían precisión matemática estricta.

8.2. Prospectiva

Con el propósito de dar continuidad a esta línea de investigación y superar las barreras técnicas identificadas, se plantea como principal vía de desarrollo futuro la transición desde el RAG vectorial plano hacia arquitecturas basadas en Grafos de Conocimiento (GraphRAG). Modelar la normativa agrícola como una red explícita de entidades legales, artículos, jerarquías y excepciones permitirá ejecutar búsquedas relacionales multi-salto, resolviendo de manera estructural la dispersión contextual observada en las consultas transversales.

Adicionalmente, se contempla la evolución del sistema hacia un esquema de RAG Agéntico (Agentic RAG), en el cual diversos agentes especializados en ruteo, cálculo numérico y verificación jurídica colaboren de forma orquestada para validar la consistencia lógica y matemática de las respuestas antes de emitir un dictamen final. Finalmente, se propone escalar la base de conocimiento no paramétrica para integrar la totalidad de los boletines oficiales autonómicos y reglamentos de la Unión Europea.

8.3. Consideraciones finales

Los hallazgos derivados de este trabajo demuestran que las arquitecturas RAG no constituyen una solución mágica que elimina por completo el riesgo de alucinación, sino un marco de control probabilístico de alta efectividad. En dominios de elevada responsabilidad como la gestión de ayudas públicas de la PAC y el POSEI, la seguridad jurídica exige trascender la generación del lenguaje para consolidar sistemas transparentes, auditables y fundamentados en metadatos granulares.

A nivel personal y profesional, la elaboración de esta investigación ha consolidado mi dominio de las arquitecturas avanzadas de Procesamiento del Lenguaje Natural y el aprendizaje profundo. Más allá del desarrollo técnico, el mayor aprendizaje ha residido en la asimilación del rigor del método científico, comprendiendo la necesidad de auditar críticamente las métricas globales para identificar las vulnerabilidades subyacentes en los sistemas de inteligencia artificial.

El recorrido a lo largo del Máster ha significado una transformación integral en mi perfil técnico, permitiéndome conectar la ingeniería de datos con la gobernanza tecnológica y la resolución de problemas reales. Esta formación me ha dotado de las herramientas necesarias para diseñar soluciones innovadoras que aporten valor tangible a la sociedad y al sector productivo.

La experiencia acumulada en la estructuración de este proyecto ha fortalecido mi percepción sobre la importancia de la función docente e investigadora. La capacidad de simplificar y transmitir conceptos matemáticos y computacionales de gran complejidad resulta indispensable en el entorno académico contemporáneo. En este sentido, considero

prioritario impulsar metodologías de enseñanza centradas en la auditoría técnica y la evaluación crítica de los modelos emergentes.

Expreso mi sincero agradecimiento al equipo tutelar por su valioso acompañamiento, paciencia y rigor académico durante el desarrollo de esta tesis. De igual forma, extendiendo mi gratitud al cuerpo docente del Máster por la excelencia de la enseñanza impartida, así como a mi familia y compañeros por su apoyo constante e incondicional en cada etapa de este proceso.

No puedo finalizar este trabajo sin manifestar mi ferviente deseo que la transformación digital de nuestro sector agrario avance decididamente hacia un modelo transparente, justo y accesible. La simplificación de las cargas burocráticas mediante tecnologías seguras constituye una deuda histórica con nuestros agricultores y ganaderos, y confío en que la aportación de este Trabajo de Fin de Máster sea un paso firme hacia la consolidación de una Inteligencia Artificial al servicio del bienestar de nuestro campo.

Apéndice A

Glosario

Adaptador: Implementación concreta de un puerto hexagonal. En este trabajo, los adaptadores de salida conectan el RAG con pgvector, el índice BM25, Ollama y el Cross-Encoder; los de entrada son las rutas FastAPI que delegan en los casos de uso.

ANN (Approximate Nearest Neighbors): Técnica de búsqueda aproximada que permite localizar de forma eficiente los vectores más similares en bases de datos de gran tamaño, reduciendo significativamente el tiempo de consulta respecto a una búsqueda exhaustiva.

Arquitectura hexagonal (Ports and Adapters): Estilo arquitectónico propuesto por Cockburn (2005) en el que el dominio de negocio no depende de frameworks ni de infraestructura. El núcleo se comunica con el exterior mediante puertos (interfaces); los adaptadores implementan esos puertos hacia HTTP, bases de datos o modelos de lenguaje.

Backpropagation (Retropropagación): Algoritmo de entrenamiento utilizado en redes neuronales artificiales que ajusta los pesos de las conexiones minimizando el error mediante el cálculo del gradiente de la función de pérdida.

Base de datos vectorial (Vector Database): Sistema especializado para almacenar, indexar y recuperar vectores de alta dimensión generados por modelos de embeddings. Facilita búsquedas semánticas rápidas mediante métricas de similitud.

PostgreSQL/pgvector: Base de datos vectorial de código abierto diseñada para aplicaciones de inteligencia artificial y sistemas RAG. Destaca por su facilidad de integración con Python y LangChain.

Chunk: Fragmento de un documento utilizado como unidad mínima de procesamiento en sistemas RAG. Cada fragmento conserva parte del contexto original para facilitar la recuperación de información.

Chunking: Proceso de dividir documentos extensos en fragmentos más pequeños y manejables antes de generar sus embeddings e indexarlos en una base de datos vectorial.

CLIP (Contrastive Language-Image Pretraining): Modelo multimodal desarrollado por OpenAI capaz de representar imágenes y texto dentro del mismo espacio vectorial, permitiendo búsquedas cruzadas entre ambos formatos.

Context Precision: Métrica utilizada para evaluar la proporción de fragmentos recupe-

radados que realmente son relevantes para responder una consulta.

Context Recall: Métrica que mide la capacidad de un sistema RAG para recuperar toda la información necesaria existente en el corpus documental.

Cosine Similarity (Similitud del coseno): Medida matemática utilizada para calcular el grado de semejanza entre dos vectores mediante el ángulo formado entre ellos.

Cross Encoder: Modelo de reranking que evalúa conjuntamente una consulta y un documento para calcular una puntuación de relevancia más precisa que los modelos bi-encoder

Docker: Plataforma de virtualización ligera basada en contenedores que permite desplegar aplicaciones de forma reproducible e independiente del sistema operativo.

Embedding: Representación numérica de alta dimensión que captura el significado semántico de palabras, frases, documentos o imágenes para permitir comparaciones matemáticas entre ellos.

Faithfulness: Métrica de evaluación que determina el grado en que una respuesta generada por un modelo está fundamentada exclusivamente en la información recuperada del contexto documental.

FAISS (Facebook AI Similarity Search): Biblioteca desarrollada por Meta para realizar búsquedas eficientes de similitud vectorial en colecciones de gran tamaño.

Fine-Tuning: Proceso mediante el cual un modelo de lenguaje previamente entrenado se adapta a una tarea específica utilizando un conjunto adicional de datos especializados.

GPT (Generative Pre-trained Transformer): Familia de modelos de lenguaje basados en la arquitectura Transformer capaces de comprender y generar texto mediante aprendizaje profundo.

GPT-4V (GPT-4 Vision): Variante multimodal de GPT que incorpora capacidades de comprensión tanto de texto como de imágenes.

GraphRAG: Arquitectura RAG que combina recuperación documental con grafos de conocimiento para representar relaciones complejas entre entidades.

Guardrails: Conjunto de reglas, validaciones y mecanismos de control implementados para evitar respuestas incorrectas, peligrosas o alucinadas generadas por modelos de lenguaje.

Hallucination (Alucinación): Fenómeno por el cual un modelo de lenguaje genera información falsa, inventada o no respaldada por los datos disponibles, presentándola como verdadera.

HNSW (Hierarchical Navigable Small World): Algoritmo de indexación utilizado en bases de datos vectoriales que organiza los vectores en grafos jerárquicos para acelerar las búsquedas por similitud.

IA (Inteligencia Artificial): Disciplina de la informática dedicada al desarrollo de sistemas capaces de realizar tareas que normalmente requieren inteligencia humana.

LangChain: Framework de desarrollo que facilita la construcción de aplicaciones basadas

en modelos de lenguaje, especialmente arquitecturas RAG y agentes inteligentes.

Layout Analysis: Técnica de visión artificial utilizada para identificar automáticamente títulos, tablas, imágenes y otros elementos estructurales dentro de documentos PDF.

LlamaIndex: Framework especializado en la indexación y recuperación documental para aplicaciones basadas en modelos de lenguaje e inteligencia artificial generativa.

LLM (Large Language Model): Modelo de lenguaje entrenado sobre enormes volúmenes de texto capaz de comprender, generar y razonar utilizando lenguaje natural.

Milvus: Base de datos vectorial de código abierto optimizada para almacenar miles de millones de embeddings con alto rendimiento.

Modelo multimodal: Modelo de inteligencia artificial capaz de procesar simultáneamente distintos tipos de información, como texto, imágenes, tablas, audio o vídeo.

Multi-Head Attention: Mecanismo de la arquitectura Transformer que permite atender simultáneamente diferentes relaciones semánticas presentes dentro de una secuencia.

NLP (Natural Language Processing): Rama de la inteligencia artificial dedicada al procesamiento automático del lenguaje humano.

NLU (Natural Language Understanding): Subdisciplina del NLP centrada en la comprensión del significado, intención y contexto del lenguaje natural.

NLG (Natural Language Generation): Área del NLP encargada de generar automáticamente texto coherente y comprensible a partir de información estructurada.

OCR (Optical Character Recognition): Tecnología que convierte imágenes o documentos escaneados en texto editable mediante reconocimiento óptico de caracteres.

Ollama: Plataforma para ejecutar modelos de lenguaje de forma local, preservando la privacidad y evitando la dependencia de servicios en la nube.

PAC (Política Agraria Común): Política de la Unión Europea destinada a apoyar el sector agrícola mediante ayudas económicas, regulación del mercado y desarrollo rural.

PDF Parsing: Proceso de extracción automática de texto, tablas e imágenes desde documentos PDF para su posterior procesamiento.

Pinecone: Servicio gestionado de base de datos vectorial diseñado para aplicaciones de inteligencia artificial y búsqueda semántica a gran escala.

POSEI (Programa de Opciones Específicas por la Lejanía y la Insularidad): Programa europeo destinado a compensar las dificultades derivadas de la insularidad y ultraperiferia en determinadas regiones agrícolas.

Puerto: Contrato (interfaz) que el dominio o los casos de uso exigen al exterior, sin conocer la tecnología concreta. En el núcleo RAG los puertos salientes cubren embeddings, LLM, repositorio de chunks, índice léxico y reranker.

Prompt Engineering: Disciplina orientada al diseño y optimización de instrucciones dirigidas a modelos de lenguaje con el fin de obtener respuestas más precisas.

PyMuPDF: Biblioteca de Python utilizada para extraer texto, imágenes y metadatos desde documentos PDF.

Qdrant: Base de datos vectorial de código abierto desarrollada en Rust, optimizada para búsquedas semánticas con filtrado avanzado de metadatos.

RAG (Retrieval-Augmented Generation): Arquitectura que combina recuperación de información documental con generación de texto mediante modelos de lenguaje para producir respuestas fundamentadas en fuentes externas.

RAG Multimodal: Evolución del RAG tradicional que incorpora recuperación y procesamiento conjunto de texto, tablas, imágenes, diagramas y otros contenidos multimodales.

RAGAS (Retrieval Augmented Generation Assessment): Framework utilizado para evaluar automáticamente sistemas RAG mediante métricas como fidelidad, precisión contextual y relevancia de las respuestas.

Re-ranking: Etapa posterior a la recuperación documental donde los resultados iniciales se reordenan según su relevancia utilizando modelos más precisos.

Self-Attention (Autoatención): Mecanismo fundamental de los Transformers que permite calcular la importancia relativa entre todos los elementos de una secuencia.

Token: Unidad mínima de información procesada por un modelo de lenguaje, que puede corresponder a una palabra completa, una parte de ella o un signo de puntuación.

Tokenización: Proceso de dividir un texto en tokens antes de ser procesado por un modelo de lenguaje.

Transformer: Arquitectura de redes neuronales presentada en 2017 basada en mecanismos de autoatención, considerada el fundamento de los actuales modelos de lenguaje de gran tamaño.

TruLens: Framework empleado para monitorizar y evaluar el rendimiento, precisión y seguridad de aplicaciones basadas en modelos de lenguaje

Vectorización: Proceso mediante el cual un documento, imagen o texto se transforma en un embedding para su almacenamiento y recuperación semántica.

Vision Transformer (ViT): Arquitectura basada en Transformers adaptada al procesamiento de imágenes mediante la división de estas en pequeños fragmentos o patches.

Bibliografía

1. (Huang, 2023) Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, et al. A survey on hallucination in large language Models. <https://doi.org/10.48550/arXiv.2401.01313>
2. (Li, 2025) Li, Y., Fu, X., Verma, G., Buitelaar, P., & Liu, M. (2025). *Mitigating hallucination in large language models (LLMs): An application-oriented survey on RAG, reasoning, and agentic systems* (arXiv:2510.24476). arXiv. <https://arxiv.org/abs/2510.24476>
3. (Zhong, 2025) Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., . . . Liu, T. (2025). A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems*, 43(2), 1–55. <https://doi.org/10.1145/3703155>
4. (Yao, y otros, 2024) Yao, Y., Duan, J., Xu, K., Cai, Y., Sun, Z., & Zhang, Y. (2024). A survey on large language model (LLM) security and privacy: The Good, The Bad, and The Ugly. *High-Confidence Computing*, 4(2), 100211. <https://doi.org/10.1016/j.hcc.2024.100211>
5. (Jurafsky, 2024) Jurafsky, D., & Martin, J. H. (2024). *Speech and Language Processing* (3rd ed., draft). Stanford University. <https://web.stanford.edu/~jurafsky/slp3/>
6. (Goodfellow, 2016) Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. <https://www.deeplearningbook.org/>
7. (Chen, 1999) Chen, S. F., & Goodman, J. (1999). An Empirical Study of Smoothing Techniques for Language Modeling. *Computer Speech & Language*, 13(4), 359–394.
8. (Mikolov, 2013) Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient Estimation of Word Representations in Vector Space. <https://arxiv.org/abs/1301.3781>
9. (Pennington, 2014) Pennington, J., Socher, R., & Manning, C. D. (2014). GloVe: Global Vectors for Word Representation. *Proceedings of EMNLP 2014*. <https://aclanthology.org/D14-1162/>
10. (Elman, 1990) Elman, J. L. (1990). Finding Structure in Time. *Cognitive Science*, 14(2), 179–211. [https://doi.org/10.1016/0364-0213\(90\)90002-E](https://doi.org/10.1016/0364-0213(90)90002-E)

11. (Hochreiter S. &, 1997) Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural computation*, 9(8), 1735-1780.
12. (Bahdanau, 2014) Bahdanau, D., Cho, K., & Bengio, Y. (2014). Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*
13. (Vaswani, 2017) Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, 30.
14. (Lewis P. P., 2020) Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459-9474.
15. (Cockburn, 2005) Cockburn, A. (2005). Hexagonal architecture. <https://alistair.cockburn.us/hexagonal-architecture/>
16. (Schwaber y Sutherland, 2020) Schwaber, K., & Sutherland, J. (2020). The Scrum Guide. Scrum.org. <https://scrumguides.org/scrum-guide.html>
17. (Gautier Izacard, 2023) Izacard, G., Caron, M., Hosseini, L., Riedel, S., Bojanowski, P., Joulin, A., & Grave, E. (2021). Unsupervised dense information retrieval with contrastive learning. *arXiv preprint arXiv:2112.09118*. Recuperado de <https://arxiv.org/abs/2112.09118>
18. (Yu, 2023) Wenhao Yu, Hongming Zhang, Xiaoman Pan, Kaixin Ma, Hongwei Wang, and Dong Yu. Chain-of-note: Enhancing robustness in retrieval-augmented language models. *arXiv preprint arXiv:2311.09210*, 2023. https://proceedings.iclr.cc/paper_files/paper/2025/Paper-Conference.pdf
19. (Reimers, 2019) Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *Proceedings of EMNLP-IJCNLP 2019*, 3982-3992. <https://aclanthology.org/D19-1410/>
20. (Xiao, 2023) Xiao, S., Liu, Z., Zhang, P., & Muennighoff, N. (2023). C-Pack: Packaged resources to advance general Chinese embedding. *arXiv preprint arXiv:2309.07597*. <https://arxiv.org/abs/2309.07597>
21. (Chen, 2024) Chen, J., Xiao, S., Zhang, P., Luo, K., Lian, D., & Liu, Z. (2024). BGE M3-Embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. *arXiv preprint arXiv:2402.03216*. <https://arxiv.org/abs/2402.03216>
22. (Wang, 2022) Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. Text embeddings by weakly-supervised contrastive pre-training. *arXiv preprint arXiv:2212.03533*, 2022.

23. (Zhang, 2025) Zhang, Y., Li, M., Long, D., Zhang, X., Lin, H., Yang, B., Xie, P., Yang, A., Liu, D., Lin, J., Huang, F., & Zhou, J. (2025). Qwen3 Embedding: Advancing text embedding and reranking through foundation models. arXiv preprint arXiv:2506.05176. <https://arxiv.org/abs/2506.05176>
24. Principles, taxonomy, challenges, and open questions. arXiv preprint arXiv:2311.05232, 2023. https://proceedings.iclr.cc/paper_files/paper/2025/file/5df5b1f121c915d8bdd00db6aac20827-Paper-Conference.pdf
25. (Zhang, 2023) Siren's song in the AI ocean: A survey on hallucination in large language models. arXiv preprint arXiv:2309.01219.
26. (Ji Z, 2023) Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., ... & Fung, P. (2023). Survey of hallucination innatural language generation. ACM computing surveys, 55(12), 1-38.
27. (James, 2024) JJmachan, Anistark, Sanjeed, Sahusiddharth: Llm hallucinations. VibrantLabs. Ragas: supercarga tu evaluación de las aplicaciones de los LLMs. 2024 https://docs.ragas.io/en/stable/concepts/metrics/available_metrics/faithfulness/#faithfulness
28. Anexos
29. Todo el código fuente desarrollado durante este Trabajo Fin de Máster, incluyendo la implementación de la arquitectura RAG multimodal, la indexación documental, el procesamiento de PDFs, la generación de embeddings, la evaluación mediante RAGAS y los scripts de despliegue, se encuentra disponible en el siguiente repositorio de GitHub: <https://github.com/SamLorenzoSanc/Arquitectura-RAG-Multimodal>